



HOTEL RESERVATION MANAGEMENT SYSTEM – “StayEase”

LECTURER NAME: AZIAH BINTI ABDOLLAH

MODULE NAME: Python Programming

MODULE CODE: CT108-3-1-PYP

SUBMISSION DATE: 11th Jan 2026

Student Name	TP number
Abdullah Shamim Naime (leader)	TP100781
Ali Akber Manji	TP100852
Dishil Atulkumar Rathod	TP091840
Ahmed Ihab Ibrahim	TP116875
Jonathan Stephen Albest	TP090512

Table of Contents

1.0 Introduction and Assumptions	4
1.1 Background and Purpose of the System.....	4
1.2 StayEase Rules	4
1.2.1 Overall Hotel Structure:.....	4
1.2.2 Room Types and Layout:	4
1.2.3 Rules and Regulations	Error! Bookmark not defined.
1.3 The Role Assumptions	6
1.3.1 Manager	6
1.3.2 Receptionist	6
1.3.3 Accountant.....	7
1.3.4 Housekeeping Rules	7
1.3.5 Guest Rules.....	9
2.0 System Design	9
2.1 Main Menu Structure (Abdullah Shamim Naime).....	10
2.2 Manager (Abdullah Shamim Naime)	11
2.3 Receptionist (Ali Akber Manji).....	13
2.4 Accountant (Dishil Atulkumar Rathod)	18
2.5 Housekeeping (Jonathan Stephen Albest).....	21
2.6 Guest (Ahmed Ihab Ibrahim)	25
3.0 Explanation of Programming Concepts Applied	26
3.1 Application of Storage Types:	26
3.2 Application of Control Types:.....	29
3.3 Application of Error Handling	34
3.4 Application of Input Validation:	38
3.5 Function Flow Explanation:	42

4.0 Screenshot of Program Execution.....	47
4.1 Main page.....	47
4.2 Manager.....	48
4.3 Receptionist.....	53
4.4 Accountant	61
4.5 Housekeeping	67
4.6 Guest.....	71
5.0 Conclusion	74
5.1 Advantages of the System.....	74
5.2 Limitations of the developed system.....	75
5.3 Suggestions for Future Improvements and System Enhancements	76
6.0 References.....	77
7.0 Appendix.....	77
7.1 Workload Matrix	77
7.2 Screenshot of Text Files Contents.....	80

1.0 Introduction and Assumptions

1.1 Background and Purpose of the System

While many hotels in Malaysia have started to use websites to deal with customers, a significant number of hotels still rely on manual processes to add and update their data depending on paper systems to register. However, these paper systems often fail when exposed to a large volume of information. This method has proven to be extremely ineffective due to delayed manual room status logging and inefficient cleaning coordination (The Edge Malaysia, 2023). Furthermore, paper invoices and receipts can easily be misplaced unlike e-voicing (Fariysa, 2024).

To replace the paper-based system, our group has introduced a menu-driven hotel management called “StayEase” which helps the managers, receptionists, accountants, housekeeping and guests.

1.2 StayEase Rules

We will implement set of hotels features. These features will help guide through how the system behaves, ensuring all data is consistent and every program can run smoothly.

1.2.1 Overall Hotel Structure:

- ❖ The hotel will hold 31 rooms in the spanning various floors.
- ❖ StayEase is located in Kuala Lumpur near an MRT.
- ❖ The reception is open 24/7 for any late check-in.

1.2.2 Room Types and Layout:

The rooms are divided into 3 main types with details of their features and furniture displayed for the customers.

- ❖ **The Standard Single/Queen (STD-S/STD-Q)**

- ◆ **Standard Single (STD-S):** A basic room with one single bed with 14 metre square area and the rate per night is 109RM per night. This room includes an air conditioner, washroom, electric kettle, cabinet and daily housekeeping.
- ◆ **Standard Queen (STD-Q):** A room with a queen size bed with 18 metres square area and the rate is 125RM per night. This room includes an air conditioner, washroom, electric kettle, 32-inch television, hairdryer, and daily housekeeping.

❖ **The Deluxe Queen/Twin/Triple (DLX-Q, DLX-T, DLX-3)**

- ◆ **Deluxe Queen (DLX-Q):** A deluxe room with one queen size bed, sofa, 20 meter squared area, and the rate per night is 159 RM. This room includes an air conditioner, washroom, electric kettle, cabinet, attached balcony, and daily housekeeping.
- ◆ **Deluxe Twin (DLX-T):** A deluxe room with one queen size bed and one single size bed. The size is 20 metre square area and the rate per night is 167 RM. This room includes an air conditioner, a washroom, an electric kettle, a cabinet, a balcony and daily housekeeping.
- ◆ **Deluxe Triple (DLX-3):** A deluxe room with 3 single size bed and sofa. The room has an area of 25 metre square and the rate per night is 189 RM. This room includes an air conditioner, washroom with pair of both shampoo and conditioner, electric kettle, cabinet and daily housekeeping, attached balcony.

❖ **The Family room (FAM)**

- ◆ **Family (FAM):** A room ideal for a family with a queen size bed and 2 single beds occupying an area of 300 square feet and rate per night is 200 RM. This room includes Air conditioner, washroom, electric kettle, cabinet, daily housekeeping, and an attached balcony.

1.2 The Role Assumptions

1.2.1 Manager

The manager side has multiple assumptions made for the following as shown below:

- ❖ **Role-based:** The manager can add, update and remove rooms. The manager can view the status of each room. The manager can create reports on occupancy rate and daily and monthly revenue.
- ❖ **Data Assumption:** All room details are stored in a text file named **rooms.txt** that follows the consistent format, which is RoomID, Room Type, Occupancy, Floor, Rate, and Status. Each Room ID is UNIQUE as shown in this example, F1STD-S1 = Floor 1, STD-S type, Room 1.
- ❖ **Input Assumptions:** All roomIDs are all stored as mentioned earlier.

Process Assumptions: rooms.txt will append or rewrite in any cases of updating, adding, or removing rooms. Occupancy will be calculated based on the room being OCCUPIED and the system report also rely on the bookings.txt and payments.txt which has CHECKED_IN and Paid status respectively.

1.2.2 Receptionist

The receptionist module has multiple assumptions that are split into four categories. The first is role-based assumptions which involve what the receptionist can do. The receptionist's main duty involves registering guests, updating guest information, check-in, check-out, and cancelling bookings. The second is data assumptions which define how data is ordered and stored within the system. These include: each guest is assigned a system-generated guest ID, guest identification may contain alphanumeric characters, room, guest, and booking data are stored in comma-separated text files, and each record is assumed to follow a consistent format. The third is input assumptions, this allows for data to be processed without error. These include dates which are assumed to be entered in a valid format (DD-MM-YYYY), phone numbers which are assumed to be strings to support country codes, and only required fields are validated for non-empty input. The final one is processing assumptions which involve the logic that the system operates on. These include early departure is considered booking cancellation, only bookings with status CHECKED_IN can be checked out or cancelled, and rooms cannot be checked into unless their status is set to AVAILABLE.

1.2.3 Accountant

The accountant's primary responsibilities include recording guest payments, generating income reports, viewing outstanding payments, and producing monthly financial summaries.

The accountant does not modify booking details, room information, guest records or housekeeping and maintenance logs, these responsibilities belong to other roles.

Data Assumptions

Each payment record is associated with an existing booking through a unique booking ID. Payment, booking, and daily activity data are stored in comma-separated text files such as payments.txt, bookings.txt, and daily_logs.txt. Each record in these files is assumed to follow a consistent format and ordering to allow correct processing by the system. Payment status values are assumed to be limited to Paid or Unpaid.

Input Assumptions

Dates entered by the accountant are assumed to follow ISO format (YYYY-MM-DD) for financial consistency. Payment amounts are assumed to be numeric values without currency symbols. Booking IDs entered by the accountant are assumed to exist in the system, and validation is performed to prevent recording payments for non-existent bookings. Only required fields are validated to ensure simplicity and compliance with assignment requirements.

Processing Assumptions

How the accounting logic operates. Only payment records with a status of PAID are included when calculating total income. Payment records with a status of UNPAID are considered outstanding and are displayed separately. Monthly financial summaries are generated by filtering payment records based on the selected month. Recording a payment does not modify booking records directly; instead, payment activities are logged in the daily activity file to maintain separation between financial and booking data.

1.2.4 Housekeeping Rules

The Housekeeping role of the system brings into reality the responsibility to record all housekeeping activities into log, such as cleaning and maintenance to ensure that all housekeepers remain updated on activities. Especially with reasons regarding the number of

rooms and floors, it is important to have proper communication between housekeepers. Using logs, we can easily record a history of data about the status of rooms maintenance and cleanliness.

Role and Process Assumptions

Housekeepers' roles are to ensure that all rooms are cleaned, and all maintenance issues are resolved. Housekeepers are required to clean all rooms on all floors daily, and after each room has been cleaned, housekeeper is required to set the room status to clean by entering the respected room number. This activity is then logged inside a text file as a history of record, and after the user enters the date of today, together with the room number, the record is successfully logged.

Input Assumptions

How the system handles data input is one part of the project to consider. This is to prevent any errors leading to crash the whole program itself, such as an incorrect room number input, or the choice required for the user to enter, which requires numeric values, was entered as an alphanumeric or alphabetical value instead and incorrect date formatting. The program must aim to make sure that proper prompt messages given to the user ensures that valid data is entered so that the program does not shut down.

Data Assumptions

Since the files have lots of records (lines) and having each of those records with different attributes, we store them all in Comma Separated Variable (CSV) format so that we can easily extract them and represent them easily throughout the code. Files store raw data such as Room Number, Dates and housekeeping comments in a consistent manner for extraction and Insertion, with assumption that these data are read linearly.

1.2.5 Guest Rules

The Guest role is responsible for interacting hotel reservations from the perspective of a guest. As the overall flow starts from viewing available rooms, then either making room reservations or cancelling reservations. Lastly, they can view billing summaries. The Guest module interacts primarily with 3 main .txt files “rooms.txt”, “bookings.txt”, and “payments.txt” while maintaining read-only access information when necessary.

Data assumptions

Guest makes the assumptions that “rooms.txt”, “bookings.txt”, and “payments.txt” text files are provided beforehand as agreed upon by the system format. Each file consists of a comma-separated structure which allows for an easy-to-use reading, writing and appending of files without corruption.

Input assumptions

The Guest, assumes that users are already provided with certain details when prompted to input correct room IDs, booking IDs and so on. The system also provides basic public information to help the user to identify what section and information are necessary for them in rows of data displayed.

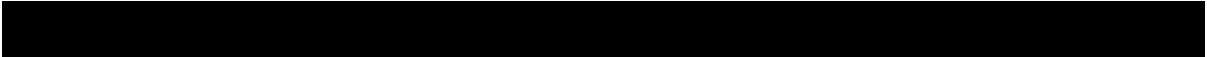
Processing assumptions

The Guest assumes basic file operations read files successfully during runtime. Booking IDs and other unique attributes are provided to ensure integrity of data.

2.0 System Design

This section presents the complete system design made by the 5 members, all compiled together, for the StayEase management. Furthermore, all our diagrams were made using the website and app design represents **Main menu structure, Role based navigation, Data Flow and File handling Logic.**

2.1 Main Menu Structure (Abdullah Shamim Naime)



START

DISPLAY "=====

DISPLAY "WELCOME TO STAY EASE HOTEL"

DISPLAY "1. Management"

DISPLAY "2. Receptionist"

DISPLAY "3. Accountant"

DISPLAY "4. Housekeeping"

DISPLAY "5. Guest"

DISPLAY "6. Exit"

DISPLAY "=====

REPEAT

 DISPLAY "Enter your choice"

 GET choice

 IF choice = 1 THEN

 DISPLAY "Manager Menu"

 ELSE IF choice = 2 THEN

 DISPLAY "Receptionist Menu"

 ELSE IF choice = 3 THEN

 DISPLAY "Accountant Menu"

 ELSE IF choice = 4 THEN

 DISPLAY "Housekeeping Menu"

 ELSE IF choice = 5 THEN

 DISPLAY "Guest Menu"

 ELSE IF choice = 6 THEN

 DISPLAY "Exiting the program"

 ELSE

 DISPLAY "Invalid choice"

 END IF

UNTIL choice = 6

END

2.2 Manager (Abdullah Shamim Naime)



START

DISPLAY "WELCOME TO MANAGEMENT"

REPEAT

 DISPLAY "1. Add Room Record"

 DISPLAY "2. Update Room Record"

 DISPLAY "3. Remove Room Record"

 DISPLAY "4. Exit Management"

 DISPLAY "Enter your choice"

 GET choice

IF choice = 1 THEN

 GET room_number

 DISPLAY "Room record added successfully"

ELSE IF choice = 2 THEN

 GET room_number

 DISPLAY "Room record updated successfully"

ELSE IF choice = 3 THEN

 GET room_number

 DISPLAY "Room record removed successfully"

ELSE IF choice = 4 THEN

DISPLAY "Exiting Management Menu"

ELSE

DISPLAY "Invalid choice"

END IF

UNTIL choice = 4

DISPLAY "SYSTEM SUMMARY"

READ all booking records

COUNT total bookings

DISPLAY total bookings

COUNT total rooms OCCUPIED

CALCULATE occupancy rate

DISPLAY occupancy rate

CALCULATE total income

DISPLAY total income

DISPLAY "MONTHLY PERFORMANCE REPORT"

FOR day FROM 1 TO 30

DISPLAY "DAY", day

READ booking records for the day

COUNT daily bookings

CALCULATE daily occupancy rate

CALCULATE daily income

DISPLAY daily summary

END FOR

2.3 Receptionist (Ali Akber Manji)


START

DISPLAY "RECEPTIONIST MENU"

WHILE TRUE

DISPLAY "-" *20

DISPLAY "Press 1 to register a new guest"

DISPLAY "Press 2 to update guest information"

DISPLAY "Press 3 to view room availability"

DISPLAY "Press 4 to check-in guest"

DISPLAY "Press 5 to check-out guest"

DISPLAY "Press 6 to cancel booking"

DISPLAY "Press 7 to exit Receptionist view"

DISPLAY "Enter your choice"

GET choice

IF choice == 7 THEN

DISPLAY "Exiting Receptionist menu"

BREAK

ENDIF

IF choice == 1 THEN

DISPLAY "REGISTER NEW GUEST"

DISPLAY "Enter guest name, ID, phone, and email:"

GET GuestName, GuestIdentification, GuestPhone, GuestEmail

IF GuestName == "" OR GuestIdentification == "" OR GuestPhone == "" OR GuestEmail == "" THEN

```
        DISPLAY "Error: Name, ID, Phone, and Email cannot be
empty."
    ELSE
        READ all guest records from "guests.txt" into guestList
        FIND highest guest ID in guestList and store in highest_ID
        SET new_guest_id = highest_ID + 1
        CREATE new guest record with new_guest_id and entered details
        APPEND new guest record to "guests.txt"
        DISPLAY "Guest registered successfully. Guest ID: ",
new_guest_id
    ENDIF
ENDIF
```

```
IF choice == 2 THEN
    DISPLAY "UPDATE GUEST INFORMATION"
    DISPLAY "Enter Guest ID to update: "
    GET search_guest_ID
    READ all guests from "guests.txt" into guestList
    SEARCH for guest with ID = search_guest_ID in guestList
```

```
IF guest not found THEN
    DISPLAY "Guest ID Not Found"
ELSE
    DISPLAY "Enter new name (leave blank to keep current)"
    DISPLAY "Enter new ID (leave blank to keep current)"
    DISPLAY "Enter new Phone (leave blank to keep current)"
    DISPLAY "Enter new Email (leave blank to keep current)"
    GET new_name, new_id, new_phone, new_email
    REPLACE old values with new values where new values are
not blank
    UPDATE that guest record inside guestList
    WRITE all updated guests records back to "guests.txt"
```

```
        DISPLAY "Guest information updated successfully."
    ENDIF

ENDIF

IF choice == 3 THEN
    DISPLAY "VIEW ROOM AVAILABILITY"
    READ all rooms from "rooms.txt" into roomList
    FOR each room in roomList
        DISPLAY RoomID, Room Type, Price, Status
    ENDFOR
ENDIF

IF choice == 4 THEN
    DISPLAY "GUEST CHECK-IN"
    DISPLAY "Enter Guest ID: "
    GET checkin_guest_id
    READ all guests from "guests.txt" into guestList
    CHECK if any guest in guestList has ID = checkin_guest_id

    IF guest does not exist THEN
        DISPLAY "Error: Guest ID Not Found, register guest first"
    ELSE
        READ all rooms from "rooms.txt" into roomList
        DISPLAY "Available rooms:"
        DISPLAY all rooms where status = AVAILABLE
        DISPLAY "Enter Room ID to assign: "
        GET selected_room_id
        CHECK if selected_room_id exists AND status = AVAILABLE in roomList

        IF room invalid OR not available THEN
            DISPLAY "Error: Invalid or unavailable room"
        ELSE
            DISPLAY "Enter Check-in date (DD-MM-YYYY): "
```

```

GET checkin_date
DISPLAY "Enter expected check-out date (DD-MM-YYYY): "
GET checkout_date
READ all bookings from "bookings.txt" into bookingList
FIND highest booking ID in bookingList and store in highest_booking_ID
SET new_booking_id = highest_booking_ID + 1
    CREATE new booking record with new_booking_id, checkin_guest_id,
selected_room_id, checkin_date, checkout_date, status = CHECKED_IN
    APPEND new booking record to "bookings.txt"
    SET status of selected_room_id to OCCUPIED in roomList
    WRITE all updated room records back to "rooms.txt"
    DISPLAY "Check-in successful. Booking ID: ", new_booking_id
ENDIF
ENDIF
ENDIF

IF choice == 5 THEN
    DISPLAY "GUEST CHECK-OUT"
    DISPLAY "Enter Booking ID: "
    GET checked_booking_id
    READ all bookings from "bookings.txt" into bookingList
    SEARCH for booking with ID = checked_booking_id in bookingList

    IF booking not found THEN
        DISPLAY "Error: Booking ID Not Found."
    ELSE IF booking status is not CHECKED_IN THEN
        DISPLAY "Error: Booking is not currently checked in"
    ELSE
        SET booking status to CHECKED_OUT in bookingList
        GET room_id linked to this booking
        READ all rooms from "rooms.txt" into roomList
        SET status of room_id to AVAILABLE in roomList
        WRITE all updated bookings records back into "bookings.txt"
    
```

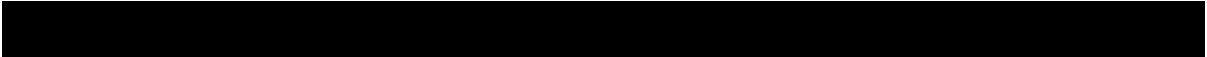
```
WRITE all updated room records back into "rooms.txt"  
DISPLAY "Check-out completed. Direct guest to Accountant for payment."  
ENDIF  
ENDIF
```

```
IF choice == 6 THEN  
  DISPLAY "CANCEL BOOKING"  
  DISPLAY "Enter Booking ID to cancel"  
  GET cancel_booking_id  
  READ all bookings from "bookings.txt" into bookingList  
  SEARCH for Booking ID = cancel_booking_id in bookingList
```

```
  IF booking not found THEN  
    DISPLAY "Error: Booking ID not found"  
  ELSE IF booking status is CHECKED_OUT THEN  
    DISPLAY "Error: Cannot cancel a checked-out booking"  
  ELSE  
    SET booking status to CANCELLED in bookingList  
    GET room_id from this booking  
    READ all rooms from "rooms.txt" into roomList  
    SET status of room_id to AVAILABLE in roomList  
    WRITE all updated booking records back to "bookings.txt"  
    WRITE all updated room records back to "rooms.txt"  
    DISPLAY "Booking cancelled successfully. Room set to AVAILABLE"  
  ENDIF  
ENDIF  
ENDWHILE
```

```
END
```

2.4 Accountant (Dishil Atulkumar Rathod)



START

DISPLAY "WELCOME TO ACCOUNTING SYSTEM"

DISPLAY "Press 1 to Record Payment"

DISPLAY "Press 2 to Generate Income Report"

DISPLAY "Press 3 to View Outstanding Payments"

DISPLAY "Press 4 to Generate Monthly Financial Summary"

DISPLAY "Press 5 to Exit Accounting System"

GET choice

WHILE TRUE

IF choice == 5 THEN

 DISPLAY "Exiting Accounting System..."

 BREAK

ENDIF

IF choice == 1 THEN

 DISPLAY "=== RECORD PAYMENT ==="

 DISPLAY "Enter Booking ID: "

 GET booking_id

 DISPLAY "Enter Payment Amount: "

 GET amount

 DISPLAY "Enter Payment Status (Paid/Unpaid): "

 GET status

DISPLAY "Enter Payment Date (YYYY-MM-DD): "

GET date_of_payment

WRITE booking_id, amount, date_of_payment, status INTO payments.file

DISPLAY "Payment successfully recorded"

ENDIF

IF choice == 2 THEN

DISPLAY "=== GENERATING INCOME REPORT ==="

READ all payment records FROM payments.file

SET total_income = 0

FOR each record IN payments.file

IF record.status == "Paid" THEN

total_income = total_income + record.amount

ENDIF

ENDFOR

DISPLAY "Total Income Collected: RM " + total_income

ENDIF

IF choice == 3 THEN

DISPLAY "=== OUTSTANDING PAYMENTS ==="

READ all payment records FROM payments.file

```
FOR each record IN payments.file
    IF record.status == "Unpaid" THEN
        DISPLAY "Booking: " + record.booking_id
        DISPLAY "Amount Due: RM " + record.amount
        DISPLAY "Date Recorded: " + record.date_of_payment
    ENDIF
ENDFOR
ENDIF
```

```
IF choice == 4 THEN
    DISPLAY "=== MONTHLY FINANCIAL SUMMARY ==="

    DISPLAY "Enter Month (YYYY-MM): "
    GET selected_month

    SET paid_total = 0
    SET unpaid_total = 0

    READ all payment records FROM payments.file

    FOR each record IN payments.file
        IF record.date_of_payment STARTS WITH selected_month THEN
            IF record.status == "Paid" THEN
                paid_total = paid_total + record.amount
            ELSE
                unpaid_total = unpaid_total + record.amount
            ENDIF
        ENDIF
    ENDFOR

    DISPLAY "Monthly Summary for " + selected_month
```

```
    DISPLAY "Total Paid: RM " + paid_total
    DISPLAY "Total Unpaid: RM " + unpaid_total
ENDIF
```

```
    DISPLAY "Enter your next choice: "
    GET choice
```

```
ENDWHILE
```

```
END
```

2.5 Housekeeping (Jonathan Stephen Albest)



```
START
```

```
    WHILE the user does not choose 4
```

```
        DISPLAY "WELCOME TO HOUSEKEEPING"
```

```
        DISPLAY "
```

1. Update Cleaning Status
 2. Report Maintenance Issue
 3. View Daily Cleaning Schedule
 4. Exit
- ```
 "
```

```
 DISPLAY "Enter your choice:"
```

GET Choice from

IF Choice = 1 THEN

    READ All records of cleaning status

    DISPLAY "Enter the room number: "

    GET room\_number

    WHILE user enters the wrong room number:

        DISPLAY "Room Number does not exist!"

        DISPLAY "Enter the room number: "

        GET room\_number

    ENDWHILE

    FOR all lines in read file

        IF room\_number = room Id from file

            SET status of room Id to Cleaned

        ENDIF

    ENDFOR

    WRITE this set of data back to file

    DISPLAY "Room cleaning status was changed to Clean"

    DISPLAY "Enter the date of today (yyyy-MM-dd): "

    GET Today\_Date

    WHILE user enters the wrong date format:

        DISPLAY "Invalid Date Format!"

        DISPLAY "Enter the date of today(yyyy-MM-dd): "

        GET Today\_Date

ENDWHILE

DISPLAY "Enter the comment: "

GET cleaning\_comments

APPEND Today\_Date, room\_number and cleaning\_comments to log

DISPLAY "Cleaning status was changed for ", Room\_Number

ELSE IF Choice = 2 THEN

DISPLAY "Enter the date of today (yyyy-MM-dd): "

GET Today\_Date

WHILE user enters the wrong date format:

    DISPLAY "Invalid Date Format!"

    DISPLAY "Enter the date of today(yyyy-MM-dd): "

    GET Today\_Date

ENDWHILE

DISPLAY "Enter the room number: "

GET room\_number

WHILE user enters the wrong room number:

    DISPLAY "Room Number does not exist!"

    DISPLAY "Enter the room number: "

    GET room\_number

ENDWHILE

DISPLAY "What seems to be the issue?"

GET maintenance\_issue

APPEND Today\_Date, room\_number and maintenance\_issue to log

DISPLAY "Maintenance issue was successfully reported"

ELSE IF Choice = 3 THEN

READ all records from Room\_Cleaning\_Status text file

FOR all lines in read file

IF room\_number is FALSE

DISPLAY room\_number and its cleaning status as FALSE

ENDIF

ENDFOR

FOR all lines in read file

IF room\_number is TRUE

DISPLAY room\_number and its cleaning status as TRUE

ENDIF

ENDFOR

ELSE:

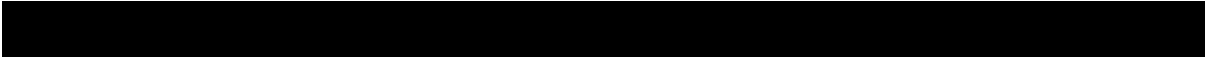
DISPLAY "Please enter a valid number"

END IF

END WHILE

END

## 2.6 Guest (Ahmed Ihab Ibrahim)

  
START

DISPLAY “Guest Menu”

1. View available rooms
2. Make a reservation
3. Cancel a reservation
4. View billing summary and payment history
5. Exit

WHILE TRUE

INPUT CHOICE

IF choice = 1 THEN

view\_available\_rooms

Else if choice = 2 THEN

make\_reservation

else if choice = 3 THEN

cancel reservation

else if choice = 4 THEN

view\_billing

else if choice = 5 THEN

print(“Exiting Guest Menu”)

break

else:

print(“Invalid choice”)

ENDWHILE

### 3.0 Explanation of Programming Concepts Applied

#### 3.1 Application of Storage Types:

##### **Manager:**

The manager side will use both list and text files when it comes to storing the data. Initially the list is used first to store the data recorded by the manager and then after it they are stored in and loaded in the rooms.txt file. This method will allow the manager to add, update and remove any type of data from the files regarding the rooms. As shown below here initially the file is read by the readlines() function then it is processed in a list by the **for line in all\_rooms:** and the **updated\_lines[]** is the new list to store the new altered data,

```
def update_room():
 print("==== UPDATE ROOM =====")
 room_to_update = input("Enter room ID to update: ").strip()
 try:
 with open("rooms.txt", "r") as f:
 all_rooms = f.readlines()
 except FileNotFoundError:
 print("rooms.txt file doesn't exist sorry")
 return
 updated_lines = []
 found = False
 for line in all_rooms:
 each_detail = line.strip().split(",")
 if each_detail[0] == room_to_update:
 current_status = each_detail[5]
```

##### **Receptionist:**

The receptionist module uses lists and text files for data storage and processing. Lists are used to temporarily read and store various records, such as guest, room, and booking data, after reading them from text files. This enables the system to search and update records effectively before overwriting the updated data back into the text files. Text files are used for long-term data storage, including data on rooms, guests, and bookings. Each record is stored in a single line using comma-separated variable form, with a unique system-generated ID used to identify and manage records.

```

search_id = input("Enter guest ID to update: ")
guests = read_file('guests.txt')
found = False

for guest in guests:
 if guest[0] == search_id:
 found = True
 new_name = input("Enter new name (leave blank to keep current): ")
 new_identification = input("Enter new ID (leave blank to keep current): ")
 new_phone = input("Enter new phone (leave blank to keep current): ")
 new_email = input("Enter new email (leave blank to keep current): ")

 if new_name != "":
 guest[1] = new_name
 if new_identification != "":
 guest[2] = new_identification
 if new_phone != "":
 guest[3] = new_phone
 if new_email != "":
 guest[4] = new_email

```

```

1,Ali Akber Manji,TP089001,0123456789,ali.manji@mail.com
2,Abdullah Shamim Naime,TP089002,0123456790,abdullah@mail.com
3,Dishil Atulkumar Rathod,TP089003,0123456791,dishil@mail.com
4,Jonathan Stephen Albrest,TP089004,0123456792,jonathan@mail.co
5,Ahmed Ihab Ibrahim,TP089005,0123456793,ahmed@mail.com
6,John Tan,MY889900,0178899001,john.tan@mail.com
7,Siti Aisyah,MY778811,0187788112,aisyah@mail.com
8,Muhammad Amir,MY667722,0166677223,amir@mail.com
9,Emily Wong,MY556633,0125566334,emily@mail.com
10,Raj Kumar,MY445544,0194455445,raj@mail.com

```

### Accountant:

The accountant module relies on text files to store and retrieve financial data such as payments, bookings, and daily transaction logs. The system reads data from files such as payments.txt, bookings.txt, and daily\_logs.txt to perform accounting operations.

When processing data, the program reads each line from the text files using file reading methods and temporarily stores the values in variables by splitting each line using commas. This allows the accountant to analyse payment records, calculate total income, and identify outstanding payments before displaying the results.

Any new payment recorded by the accountant is appended directly into the payments.txt file, ensuring that financial data is permanently stored and can be accessed again when generating reports or summaries. This approach ensures data persistence and complies with the assignment requirement of using file-based storage instead of databases.

```
def record_payment(): 1 usage
 print("\n=== RECORD PAYMENT ===")

 booking_id = input("Enter Booking ID: ")

 booking_found = False
 try:
 with open(BOOKINGS_FILE, "r") as file:
 for line in file:
 if line.startswith(booking_id + ","):
 booking_found = True
 break
```

### Housekeeping:

The housekeeping part of the project uses tuples, lists, text files and dictionaries to store data with consideration of data integrity, structure of storage, volatility and mutability of data storage. The use of text files allows us to permanently store data such as room cleaning statuses, cleaning logs and maintenance in a comma separated value (CSV) format.

To extract these values from the file for accessibility and alterability, the files are read as single records and assigned inside a dictionary. This forms the data in each line of the file in a more structured way so that it can be easily manipulated in a key- Value form.

Since a file can have more than one record, the extracted records stored in those dictionaries are then stored inside a list. This way, values stored inside that list- which are immutable, can be altered or referenced easily. In other words, the list will contain all the lines inside the file, ready for manipulation.

Finally, using tuples, values to be stored inside the text files are stored inside these tuples making it read-only. This makes sure that this data is not manipulated, promoting data integrity.

```

def return_key_values(file_path, tuple_of_keys): 5 usages
 list_of_values = [] # Empty list

 try:
 with open(file_path, 'r') as housekeeping_file: # File opened for read
 list_of_records = housekeeping_file.read().splitlines()
 for record in list_of_records:
 dictionary_of_values = { } # Empty dictionary

 # Code below assigns the values in each record to its matching Key Pairs from the Keys inside the tuple
 for i in range(0, len(tuple_of_keys)): # Loops through the tuple to reference Keys

 tuple_key = tuple_of_keys[i] # Key
 value = record.split(',')[i] # Value
 dictionary_of_values.update({tuple_key:value}) # Uses the keys from the tuple and assigns the value

 list_of_values.append(dictionary_of_values) # Appends the dictionary to the list of values
 except FileNotFoundError:
 print(f"File at {file_path} was not found")
 return list_of_values

```

**Guest:**

Uses text files as permanent storage of data to maintain reference across program executions. The 3 primary files being rooms.txt, bookings.txt, and payments.txt which store all information about the rooms, bookings, and billing details respectively. They are also temporarily stored as variables and lists for processing into display.

```

111 #3. Cancelling reservation
112 def cancel_reservation(): 1 usage
113 print("Cancel reservation(s)")
114
115 booking_ID = input("Enter your booking ID: ")
116 while not booking_id_validation(booking_ID):
117 booking_ID = input("\nPlease enter a valid booking ID: ")
118 print("Booking ID is", booking_ID)
119
120 roomID = ""
121 list_of_bookings = []
122 list_of_rooms = []
123
124 try:
125 with open("bookings.txt", "r") as f:
126 bookings = f.read().splitlines()
127 for booking in bookings:
128 record = booking.split(",")
129 if record[0] == booking_ID:
130 record[5] = "CHECKED_OUT"
131
132 roomID = record[2]
133 list_of_bookings.append(",".join(record))
134
135 with open("bookings.txt", "w") as f:
136 for record in list_of_bookings:
137 f.writelines(record + "\n")
138 except FileNotFoundError:
139 print("No bookings file can be found")
140

```

**3.2 Application of Control Types:****Manager:**

The manager part of code also uses the control statements the if, Elif and else to give the manager the options example the option to choose from altering the current rooms, view summary and view the daily and monthly reports as well. Furthermore, the while loop is used to ensure the Manager keeps on working till the manager decided to exit and thus the break function is used Example shown below the if, Elif and else is used for giving manager to choose and while loop. Each choice inputted by the user will call in a specific function.

```
def Manager_Menu():
 print("""WELCOME TO MANAGER PROFILE
 1. MANAGE ROOM RECORDS
 2.VIEW SYSTEM SUMMARY
 3.GENERATE DAILY AND MONTHLY PERFORMANCE REPORT
 4.EXIT""")
 while True:
 Choice = int(input("Enter your choice for the following command: "))
 if Choice == 4:
 print("exiting the program")
 break
 elif Choice == 3:
 view_report()
 elif Choice == 2:
 view_summary()
 elif Choice == 1:
 manager_menu()
 else:
 print("Invalid choice please try enter another value")
```

### Receptionist:

The receptionist module mainly makes use of selective statements such as if and elif, which are used for menu selection handling, determining which program branch executes runs based on choices given to the user. When a given choice is entered only the corresponding branch executes while all others are skipped.

```

34
35 > if choice == '7': ...
38
39 > elif choice == '1': ...
57
58 > elif choice == '2': ...
86
87 > elif choice == '3': ...
92
93 > elif choice == '4': ...
133
134 > elif choice == '5': ...
157
158 > elif choice == '6': ...

```

The receptionist module makes use of while loops and for loops. A while loop is used to keep the receptionist menu running continuously preventing the program from exiting after one action. The while loop is paired with an if statement that contains a break to control loop termination thereby avoiding infinite execution. For loops perform iteration, with selection occurring using if statements inside the loop. Iteration is used to search or display records, while selection is used to decide whether a specific operation should be executed.

```

while True:
 print("-"*20)
 print("Press 1 to register a new guest")
 print("Press 2 to update guest information")
 print("Press 3 to view room availability")
 print("Press 4 to check-in guest")
 print("Press 5 to check-out guest")
 print("Press 6 to cancel booking")
 print("Press 7 to exit Receptionist menu")

```

```

for guest in guests:
 if guest[0] == search_id:
 found = True
 new_name = input("Enter new name (leave blank to keep current): ")
 new_identification = input("Enter new ID (leave blank to keep current): ")
 new_phone = input("Enter new phone (leave blank to keep current): ")
 new_email = input("Enter new email (leave blank to keep current): ")

 if new_name != "":
 guest[1] = new_name
 if new_identification != "":
 guest[2] = new_identification
 if new_phone != "":
 guest[3] = new_phone
 if new_email != "":
 guest[4] = new_email

```

**Accountant:**

The accountant module makes extensive use of control statements such as if, elif, and else to manage user choices within the accounting system. These statements allow the accountant to select different operations such as recording payments, generating income reports, viewing outstanding payments, or producing monthly financial summaries.

A while loop is used to keep the accounting system running continuously until the accountant chooses to exit. The break statement is then used to terminate the loop when the exit option is selected. Each menu choice entered by the user triggers a specific function.

```
def accountant_menu():
 while True:
 print("\n=== ACCOUNTING SYSTEM ===")
 print("1. Record Payment")
 print("2. Generate Income Report")
 print("3. View Outstanding Payments")
 print("4. Generate Monthly Financial Summary")
 print("5. Exit Accounting System")

 choice = input("Enter your choice: ")

 if choice == "1":
 record_payment()
 elif choice == "2":
 generate_income_report()
 elif choice == "3":
 view_outstanding_payments()
 elif choice == "4":
 monthly_financial_summary()
 elif choice == "5":
 print("Exiting Accounting System...")
 break
 else:
 print("Invalid choice. Try again.")
```

**Housekeeping:**

Using control statements such as decision making if statements, the housekeeping module can easily navigate through the menu and validate data entered by the user. Specifically, the main menu, which required choices made by the user to execute assigned statements, displaying not cleaned rooms, and promoting the efficiency of this program, making sure values that do not require altering remain the same.

Moreover, conditional loops and iteration construct such as while and for loops were used to prevent the program from stopping until the user intentionally faces the function's sentinel value, manoeuvring through data storages, and validating important data such as date formats and room numbers.

These control statements are crucial to the project's operation, allowing the project to have a streamlined flow.

```
def housekeeping_menu(): 1 usage

 refresh_existing_rooms()

 while True:
 try: # Exception prevents program from crashing if user does not enter an integer

 # The code below displays the main menu
 print("\n\nWELCOME TO HOUSEKEEPING")
 print("=" * 40)
 print("1. Update Cleaning Status")
 print("2. Report Maintenance Issue")
 print("3. View Daily Cleaning Schedule")
 print("4. Exit")
 print("=" * 40)

 choice = int(input("Enter your choice: "))

 if choice == 1:...

 elif choice == 2:...

 elif choice == 3:...

 elif choice == 4:...

 else:...

 except ValueError:
 print("Please use numbers to select your choice!")
```

**Guest:**

Selection control structures like If, Elif, and else statements are used throughout the program to execute the filtering, and execution of any function. Meanwhile iteration loops like 'while' are used repeatedly within both the Guest menu systems and separate functions until they choose to exit the program.

```

233 def guest_menu(): 1 usage
237 print("\n*40","\n")
238 print("Welcome to The Guest Menu")
239 print("1. View available rooms")
240 print("2. Make a reservation")
241 print("3. Cancel a reservation")
242 print("4. View billing summary")
243 print("5. Exit")
244
245 choice = int(input("Enter your choice: "))
246 print("")
247
248 if choice == 1:
249 available_rooms()
250 elif choice == 2:
251 make_reservation()
252 elif choice == 3:
253 cancel_reservation()
254 elif choice == 4:
255 view_billing()
256 elif choice == 5:
257 print("Exiting Guest Menu")
258 break
259 else:
260 print("Invalid choice")
261 except ValueError:
262 print("Please enter a numeric value only")
263
264 guest_menu()

```

### 3.3 Application of Error Handling

**Manager:**

The manager uses the 2-error handling approaches one is the try except which deals with the FileNotFoundError when certain file doesn't not exist thus prevent future crash out and showcasing a more friendly line than dishing out many lines of errors. 2<sup>nd</sup> is the one that deals with dealing invalid inputs example non-existent room ID written

```

if not found:
 print("Room not found.")
 return

```

```

try:
 with open("rooms.txt", "r") as f:
 lines = f.readlines()
except FileNotFoundError:
 print("rooms.txt file doesn't exist sorry")
 return

```

### Receptionist:

The receptionist module makes use of two methods of error handling. The first and explicit method is the use of try/except. This is specifically used when encountering the FileNotFoundError when reading text files, thereby preventing program crashes when files are missing. The second and logical is logical error handling using conditional statements. This detects incorrect system requests, stops incorrect actions, and displays user-friendly error messages such as room not available and guest ID not found during receptionist operations.

```

def read_file(filename):
 data = []
 try:
 with open(filename, 'r') as file:
 for line in file:
 data.append(line.strip().split(','))
 except FileNotFoundError:
 return []
 return data

```

```

if not found:
 print("Error: Guest ID not found.")

```

**Accountant:**

The accountant module applies error handling using try and except blocks to manage potential runtime errors. This is mainly used to handle `FileNotFoundError`, which may occur if a required text file does not exist. Instead of causing the program to crash, the system displays a clear and user-friendly error message.

Logical error handling is also applied to manage invalid inputs, such as non-existent booking IDs or incorrect menu selections, this improves system reliability.

```
try:
 with open(PAYMENTS_FILE, "r") as file:
 for line in file:
 booking_id, amount, date, status = line.strip().split(",")
 if status == "Paid":
 total_income += int(amount)
except FileNotFoundError:
 print("Payments file not found.")
return
```

**Housekeeping:**

Housekeeping project has kept in mind, two types of errors, and one of them is the 'FileNotFound' error, occurred when either the file path is incorrect or file does not exist, and the other error which is 'ValueError' which occurs when a data type cast has failed.

These errors, if unhandled, could crash the program; therefore, exception handling is implemented to keep the program running, therefore making it impossible for the program to stop only until the user intentionally exits.

These exception handling methods were used especially when handling and accessing text Files, so the program runs nevertheless, and when validating user inputs making sure the type casting of user input is valid.

```
def return_key_values(file_path, tuple_of_keys): 5 usages
 list_of_values = [] # Empty list

 try:
 with open(file_path, 'r') as housekeeping_file: ...
 except FileNotFoundError:
 print(f"File at {file_path} was not found")
 return list_of_values
```

```
def housekeeping_menu(): 1 usage

 refresh_existing_rooms()

 while True:
 try: ...

 except ValueError:
 print("Please use numbers to select your choice!")
```

**Guest:**

Try-except blocks are used to prevent the program from crashing if a file is missing or cannot be opened. In guest's coding, it reads rooms.txt, reads & writes bookings.txt, and reads payments.txt. If eventually stumbled onto an error, the user can at least be returned to the main Guest menu so it allows the system to continue flowing smoothly instead of forcing the program to terminate abruptly.

```

try:
 with open("rooms.txt", "r") as f:
 for x in f:
 room = x.strip().split(",")

 if room[5] == "AVAILABLE":
 found = True
 print(
 "| Room ID: ", room[0], "\t"
 "| Occupancy:", room[2], "\t\t",
 "| Price:", room[4], "\t\t",
)
except FileNotFoundError:
 print("No rooms file can be found")
 return

if not found:
 print("Sorry, no available rooms can be found")

```

### 3.4 Application of Input Validation:

#### **Manager:**

The manager side of code ensures that the user writes the inputs Example shown here the status of the rooms will be restricted to only 3 options 1 being Available 2 being Under maintenance and 3 being OCCUPIED this is needed to prevent the user to write wrong or misleading statuses. The user will have to just type the number rather than writing long words that maybe be written wrong if careless

```

print(f"Room {room_to_update} found. Current status: {current_status}")
print("PLEASE ALTER THE ROOM STATUS based on the liking:")
print("1. AVAILABLE")
print("2. UNDER MAINTENANCE")
print("3. OCCUPIED")
change_option = input("Enter your choice (1/2/3): ").strip()

if change_option == "1":
 new_status = "AVAILABLE"
elif change_option == "2":
 new_status = "UNDER MAINTENANCE"
elif change_option == "3":
 new_status = "OCCUPIED"
else:
 print("Wrong choice please put a valid choice.")
 return

```

### Receptionist:

The receptionist module applies input validation in multiple areas. One area implemented checks if name, identification, phone, and email are entered when registering a new guest. This ensures that the system reduces the risk of incomplete or empty records stored, preventing invalid guest records from being written into guests.txt.

```

if name == "" or identification == "" or phone == "" or email == "":
 print("Error: All fields are required.")

```

### Accountant:

The accountant module uses basic input validation to prevent incorrect data entry. For example, the payment amount is checked to ensure that only numeric values are accepted, and the payment status is restricted to valid options such as “Paid” or “Unpaid”. The booking ID entered by the accountant is validated against bookings.txt to ensure that payments are only recorded for existing bookings.

These help maintain data integrity and reduce the chances of errors in financial records.

```

if not booking_found:
 print("Booking ID not found.")
 return

amount = input("Enter Payment Amount: ")
while not amount.isdigit():
 amount = input("Invalid amount. Enter numbers only: ")

status = input("Enter Payment Status (Paid/Unpaid): ").capitalize()
while status not in ["Paid", "Unpaid"]:
 status = input("Enter Paid or Unpaid only: ").capitalize()

```

### Housekeeping:

The housekeeping module contains two main input validation, having the first one being the room number validation to make sure that the room number entered by the user exists or not, and the second validation makes sure that the date format (YYYY-MM-DD) is correct. If both inputs are invalid, the user will be asked to enter a different value again.

These validation processes have been broken down into two different Functions returning True if the input is valid and false whether it's not. They are structured this way because throughout the program, users will be required to enter these values, and for efficiency the program just calls the function.

```

def room_validation(room_id): # Room Number passed as an argument 2 usages

 # The line below gets file records as a list of dictionaries
 records = return_key_values(file_path: 'Room_Cleaning_Status.txt', tuple_of_keys: ('RoomId', 'Cleaned'))

 for record in records:
 if record['RoomId'] == room_id: # Compares records of rooms with passed room number
 return True
 return False

```

```

def date_validation(date): 2 usages
 try: # Exception to catch value error if integer cast fails
 if len(date) == 10 and int(date[0:4]) and int(date[5:7]) and int(date[-2:]) and date.count('-') == 2:
 return True
 else:
 return False
 except ValueError:
 return False

```

```
The code below calls send_to_cleaning_log() to log this change to the cleaning log
today_date = input('Enter the date of today (yyyy-MM-dd): ') # Asks user for valid date
while not date_validation(today_date):
 today_date = input('Invalid Date Format!\nEnter the date of today(yyyy-MM-dd): ')

room_number = input('Enter the room number: ') # Asks user for valid Room Number
while not room_validation(room_number):
 room_number = input('Room Number does not exist!\nEnter the room number: ')

```

### Guest:

Input validation is applied to ensure user inputs fit the format as expected within the text files. This overall prevents logical errors and incorrect file updates and system crashes. Some forms of validation are pre-emptively made as a function for key attributes like Guest ID, Booking ID, dates and so on.

```
VALIDATION code for entering specific data
> def date_validation(date):...
> def room_validation(guest_ID):...
> def payment_id_validation(payment_id):...
> def booking_id_validation(booking_id): 1usage
> try:
> with open("bookings.txt", "r") as f:
> lines = f.read().splitlines()
> for record in lines:
> if record.split(',')[0] == booking_id:
> return True
> except FileNotFoundError:
> print("bookings.txt file not found")
> return False

```

These functions can later be called upon to ensure smoothness.

```
67
68 selected_room = input("\nTo reserve, please enter your room ID: ")
69 while not room_validation(selected_room):
70
71 selected_room = input("\nPlease enter a valid room ID: ")
72 print("You selected: ", selected_room)
73
74 # Get check-in/check-out dates
75 checkin_date = input("Enter check-in date (YYYY-MM-DD): ")
76 while not date_validation(checkin_date):
77 checkin_date = input("\nInvalid date format\nPlease enter a valid date (YYYY-MM-DD): ")
78
79 checkout_date = input("Enter check-out date (YYYY-MM-DD): ")
80 while not date_validation(checkout_date):
81 checkout_date = input("\nInvalid date format\nPlease enter a valid date (YYYY-MM-DD): ")
82

```

### 3.5 Function Flow Explanation:

#### **Manager**

##### **1. Add Room**

- a. The manager can add rooms
- b. The manager must write floor number; room type and room variant
- c. The system will then generate the number based on the manager
- d. Then the new room is added to the rooms.txt
- e. Thus, updated in the rooms.txt file

##### **2. Update Room**

- a. The manager enters the Room ID
- b. Once the Room is found in the rooms.txt file
- c. Then the status of the Room can be change to UNDER MAINTAINACE, AVAILABLE or OCCUPIED (1-3 status)
- d. System updates the rooms.txt file
- e. Room will successfully be updated or will show Room not found if not found

##### **3. Remove Room**

- a. The manager enters the room ID
- b. Then the room is found in the rooms.txt file
- c. The system rewrites the entire thing without the remove room
- d. System will either confirm the deletion or notify no Room found

##### **4. View Rooms**

- a. The manager can view the rooms.txt file
- b. System will then show the entire the file

##### **5. View System Summary**

- a. The manager can view the bookings.txt file and reads the CHECKED\_IN status
- b. System calculates the occupancy rate by number of checked in rooms / total rooms \* 100
- c. System reads the paymnets.txt file and makes combined total income
- d. System will only include the one with payment status Paid.

##### **6. Make Reports**

- a. The Manager can choose either make the monthly or daily reports

- b. The System reads the files and counts the CHECKED\_IN and Paid from paymets.txt and bookings.txt respectively
- c. Thus, the system will show both income and occupied that month or day

## **Receptionist**

### **1. Register guest**

- a. Receptionist selects Register Guest by pressing 1
- b. System requests guest details
- c. System checks whether required fields are filled
- d. System generates a new guest ID
- e. System saves guest details to guests.txt
- f. System sends confirmation message

### **2. Update Guest Information**

- a. Receptionist enters guest ID
- b. System searches for the matching guest ID in guests.txt
- c. System requests updated fields
- d. System replaces existing values only if a new input is given
- e. System writes updated data into guests.txt
- f. System displays confirmation message or an error

### **3. Check-in Guest**

- a. Receptionist enters guest ID
- b. System verifies guest exists in guests.txt
- c. Receptionist selects a room from the available list
- d. System validates the availability of selected room
- e. System requests a check-in date and check-out date
- f. System creates booking and saves it to bookings.txt
- g. System updates room status
- h. System sends confirmation message

### **4. Check-out/Cancel Booking**

- a. Receptionist enters booking ID
- b. System validates booking status in bookings.txt
- c. System updates booking status
- d. System frees corresponding room
- e. System updates relevant files

- f. System displays confirmation or error message

**Accountant:****1. Record Payment**

- a. The accountant can record a payment made by a guest.
- b. The accountant must enter the Booking ID, payment amount, payment status (Paid or Unpaid), and date of payment.
- c. The system will first check whether the entered Booking ID exists in the bookings.txt file.
- d. If the Booking ID is valid, the system writes the payment details into the payments.txt file.
- e. At the same time, the system records the transaction into the daily\_logs.txt file for tracking and auditing purposes.
- f. The system then confirms that the payment has been successfully recorded or displays an error message if the booking is not found.

**2. Generate Income Report**

- a. The accountant can generate the hotel's total income report.
- b. The system reads all records from the payments.txt file.
- c. Only payment records with the status Paid are selected.
- d. The system calculates the total income by summing all paid payment amounts.
- e. Finally, the system displays the total income collected by the hotel.

**3. View Outstanding Payments**

- a. The accountant can view all outstanding payments.
- b. The system reads all payment records from the payments.txt file.
- c. Records with the payment status Unpaid are filtered.
- d. The system displays the Booking ID, amount due, and date recorded for each unpaid payment.
- e. If no unpaid payments are found, the system notifies the accountant accordingly.

#### **4. Generate Monthly Financial Summary**

- a. The accountant can generate a monthly financial summary.
- b. The accountant enters the required month in the format YYYY-MM.
- c. The system reads all payment records from the payments.txt file.
- d. The system checks which payment records belong to the selected month.
- e. Payments are separated into Paid and Unpaid categories.
- f. The system calculates the total paid amount and total unpaid amount for that month.
- g. Finally, the system displays the monthly financial summary showing both totals.

#### **5. Exit Accounting System**

- a. The accountant can choose to exit the accounting system.
- b. The system displays an exit message.
- c. The loop is terminated, and control is returned to the main system menu.

### **Housekeeping:**

#### **1. Send to Maintenance Log:**

- a. Housekeeping can report any maintenance issues
- b. Housekeeping required to enter today's date and a valid room number
- c. The system validates room number and that the date is in the Format (yyyy-MM-dd)
- d. The system asks the housekeeper what the issue is
- e. The system combines data and records it in Maintenance\_Log.txt
- f. The system reports back to housekeeping whether the operation was successful, or the file was not found

#### **2. Set Cleaning Status:**

- a. Housekeeping can update the cleaning status of the room.
- b. The system reads the existing statuses from Room\_Cleaning\_Status.txt
- c. The system asks housekeepers to enter a valid room number and validates the user input.

- d. The system searches through read files for the status of given room number
- e. If the status is False (Unclean), then the system changes the record to true (clean)
- f. The system records this new set of data to the file and if successful, the system displays success message.
- g. Housekeeping is prompt to enter today's date, and the system validates the answer
- h. The system sends this change of status to the Cleaning\_Log.tx

### **3. Get Daily Cleaning Schedule**

- a. Housekeeping can view a list of rooms with – the status 'Cleaned' and/or 'Not Cleaned'
- b. The system reads through Room\_Cleaning\_Status.txt
- c. The system first displays all rooms which are not cleaned and lastly displays all the rooms that are cleaned.

#### **Guest:**

##### **1) Guest Menu Flow**

- a) Starts by displaying the options
- b) The user is able to click any option from 1-5
- c) Corresponding function is called when inputted a number
- d) If the input is invalid/does not match, it displays an invalid input, further clarifying with validation.

##### **2) View Available rooms**

- a) Called function to view available rooms
- b) Rooms.txt is first read
- c) Data is filtered and extracted RoomID, occupancy, and price. Additionally includes validation for when no room files can be found
- d) Relevant room details are then displayed

##### **3) Make Reservation**

- a) Available rooms are automatically displayed to the Guest
- b) The user can choose a room with the valid and specific room ID

- c) The user is then prompted for a check-in and check-out date
- d) Before successfully making a reservation, a booking ID is generated accordingly from the bookings.txt file after reading for the latest booking ID that was reserved.
- e) The room status is then updated to OCCUPIED.

#### 4) **Cancel reservation**

- a) User is prompted to enter a booking ID.
- b) Matching booking ID will be referred to bookings.txt.
- c) Booking status is then cancelled.
- d) Hence, the corresponding room status will be updated to “AVAILABLE”.

#### 5) **View Billing Summary**

- a) User is prompted to enter a booking ID
- b) The system reads the payments.txt
- c) Relevant data such as Amount, Date, and Status are shown from the common key attribute “Booking ID”.
- d) If relevant data are not found, an error message will be displayed.

### 4.0 Screenshot of Program Execution

#### 4.1 Main page

The main page asks user the choose between the 5 roles as shown as shown in the output below. As shown in the below writing the 1 takes the user to the Manager Menu.

```

=====
WELCOME TO THE STAY EASE HOTEL
 1. Management
 2. Receptionist
 3. Accountant
 4. Housekeeping
 5. Guest
 6. Exit
=====
Enter your choice for the following command: 1
HELLO
WELCOME TO MANAGER MENU
 1. MANAGE ROOM RECORDS
 2.VIEW SYSTEM SUMMARY
 3.GENERATE DAILY AND MONTHLY PERFORMANCE REPORT
 4.EXIT
Enter your choice for the following command:

```

## 4.2 Manager

The manager menu gives the user 3 main functionalities which are managing the room records, viewing the system summary and making the monthly and daily reports. As shown below the user is taken to the room management menu

```

WELCOME TO MANAGER MENU
 1. MANAGE ROOM RECORDS
 2.VIEW SYSTEM SUMMARY
 3.GENERATE DAILY AND MONTHLY PERFORMANCE REPORT
 4.EXIT
Enter your choice for the following command: 1

===== MANAGER ROOM MANAGEMENT =====
1. Add Room
2. Update Room
3. Remove Room
4. View Rooms
5. Exit
Enter your choice to change the rooms: |

```

## Managing Room Records

### 1) Add Rooms

```

===== MANAGER ROOM MANAGEMENT =====
1. Add Room
2. Update Room
3. Remove Room
4. View Rooms
5. Exit
Enter your choice to change the rooms: 1
===== ADD ROOM =====
Enter the floor from 1 to 4: 2
Enter room type (STD-S / STD-Q / DLX-Q / DLX-T / DLX-3 / FAM) to add: STD-S
Enter room number: 8
Room F2STD-S8 added to the rooms.txt file

```

The Screenshot shows the Add room feature under the manager role. The manager will write details floor number room type and the room number. After the system then updates the file system and writes the new room in the rooms.txt files and giving a confirmation of adding the room by printing a successful message. The room is successfully added to the file as shown below

```

F3STD-S67,STD-S,Single,3,109,AVAILABLE
F3FAM3,FAM,Family,3,200,AVAILABLE
F2STD-S2,STD-S,Single,2,109,AVAILABLE
F2STD-S8,STD-S,Single,2,109,AVAILABLE

```

## 2) Update Rooms

```

Enter your choice to change the rooms: 2
===== UPDATE ROOM =====
Enter room ID to update: F2STD-S8
Room F2STD-S8 found. Current status: AVAILABLE
PLEASE ALTER THE ROOM STATUS based on the liking:
1. AVAILABLE
2. UNDER MAINTENANCE
3. OCCUPIED
Enter your choice (1/2/3): 3
Room status updated successfully to the room.txt file

```

The screenshot shown above shows the update room feature in which the manager will write combined room name. After it the room is found in the rooms.txt file and its status is changed based on the user. The status of the room is changed as shown below

```
F2STD-S8,STD-S,Single,2,109,OCCUPIED
```

### 3) Remove Rooms

```
Enter your choice to change the rooms: 3
Enter Room ID to remove (e.g., F1STD-S1): F2STD-S8
F2STD-S8 removed.
```

Room can be removed by exactly writing the room name and once found by the system the room is removed and updated in the rooms.txt files as proved below as now the F2STD-S8 is now removed from the files

```
F3FAM3,FAM,Family,3,200,AVAILABLE
F2STD-S2,STD-S,Single,2,109,AVAILABLE
F2FAM2,FAM,Family,2,200,AVAILABLE
```

## Viewing the System Summary

### 1) Total Bookings

```

===SYSTEM SUMMARY===
=====
press the following to view the system summary:
 1.total booking
 2.occupancy rate
 3.income.
 4.to EXIT
Enter your choice for the following: 1
=== VIEW TOTAL BOOKING ===
TOTAL BOOKING: 6
check out: 4
=====

```

The system reads bookings.txt and counts rooms occupied which has CHECKED\_IN status in the files and tells which rooms are check out as well as shown below

```

2001,1,F1STD-S2,2025-12-20,2025-12-22,CHECKED_OUT
2002,2,F1DLX-Q1,2025-12-22,2025-12-26,CHECKED_IN
2003,3,F1FAM1,2025-12-21,2025-12-25,CHECKED_IN
2004,4,F2STD-S2,2025-12-23,2025-12-24,CHECKED_OUT
2005,5,F2DLX-Q1,2025-12-24,2025-12-27,CHECKED_IN
2006,6,F3STD-Q1,2025-12-20,2025-12-23,CHECKED_OUT
2007,7,F3DLX-T1,2025-12-22,2025-12-26,CHECKED_IN
2008,8,F4DLX-Q1,2025-12-24,2025-12-28,CHECKED_IN
2009,9,F2DLX-32,2025-12-23,2025-12-25,CHECKED_OUT
2010,10,F3FAM1,2025-12-24,2025-12-29,CHECKED_IN|

```

## 2) Occupancy Rate

```

Enter your choice for the following: 2
=== VIEW OCCUPANCY ===
=====
TOTAL ROOMS : 32
OCCUPIED ROOMS : 12
OCCUPANCY RATE : 37.50%
=====
=====

```

The file system reads the room text files and reads how many rooms and then counts the rooms have OCCUPIED status and calculate the rate by occupancy rate = rooms that are occupied / total rooms \* 100

### 3) Income

```

press the following to view the system summary:
 1.total booking
 2.occupancy rate
 3.income.
 4.to EXIT
Enter your choice for the following: 3
=== INCOME CHECK ===
TOTAL INCOME: 2076.0
=====

```

The system reads the payments.txt and counts the payment in which the payment status is Paid, and all the income per rooms are all added up and thus getting the combined final income

## Generate Reports

### 1) Generate Daily

```

PRESS FOR THE FOLLOWING PLEASE
 1 FOR DAILY REPORTS
 2 FOR MONTHLY REPORTS
 3 FOR EXIT
Enter Choice for: 1
Enter the date (YYYY-MM-DD): 2025-12-22

===== DAILY REPORT: 2025-12-22 =====
Total Bookings: 2
Total Income (RM): 443.00

```

The date of 22 December 2025 is added, and the system reads the bookings.txt file and count only the active check in rooms and then check the payment that has the status Paid only

## 2) Generate Monthly

```

PRESS FOR THE FOLLOWING PLEASE
 1 FOR DAILY REPORTS
 2 FOR MONTHLY REPORTS
 3 FOR EXIT
Enter Choice for: 2
Enter the month (YYYY-MM): 2025-11

===== MONTHLY REPORT: 2025-11 =====
Total Bookings: 3
Total Income (RM): 560.00
PRESS FOR THE FOLLOWING PLEASE
 1 FOR DAILY REPORTS
 2 FOR MONTHLY REPORTS
 3 FOR EXIT

```

Same here the month of November is written, and the system reads the files both the booking and payment and only counts if the status is both CHECKED\_IN and Paid respectively.

## 4.3 Receptionist

As shown below, the receptionist's module consists of six major functions: registering guests, updating guest information, viewing room availability, checking in guests, checking out

guests, and cancelling bookings. The final option exits the receptionist module and returns to the main page.

```
print('RECEPTIONIST MENU')
while True:
 print("-"*20)
 print("Press 1 to register a new guest")
 print("Press 2 to update guest information")
 print("Press 3 to view room availability")
 print("Press 4 to check-in guest")
 print("Press 5 to check-out guest")
 print("Press 6 to cancel booking")
 print("Press 7 to exit Receptionist menu")
```

### 1. Registering guests

The system prompts the receptionist to enter the guest's details, including their name, identification (such as a passport number), phone number, and email. The system then validates that none of these fields are left empty before registering the details into the text file `guests.txt`, alongside an identification number that is updated within the code to ensure that the latest guest has the highest ID number.

## RECEPTIONIST MENU

```

Press 1 to register a new guest
Press 2 to update guest information
Press 3 to view room availability
Press 4 to check-in guest
Press 5 to check-out guest
Press 6 to cancel booking
Press 7 to exit Receptionist menu
Enter your choice: 1
REGISTER NEW GUEST
Enter guest name: Messi
Enter guest ID: TAE1920
Enter guest phone number: +1245546445
Enter guest email: messiiscool@gmail.com
Guest registered successfully with Guest ID: 12

```

```

7 attachments > [icon] guests.txt
1 1,Ali Akber Manji,TP089001,0123456789,ali.manji@mail.com
2 2,Abdullah Shamim Naime,TP089002,0123456790,abdullah@mail.com
3 3,Dishil Atulkumar Rathod,TP089003,0123456791,dishil@mail.com
4 4,Jonathan Stephen Albest,TP089004,0123456792,jonathan@mail.com
5 5,Ahmed Ihab Ibrahim,TP089005,0123456793,ahmed@mail.com
6 6,John Tan,MY889900,0178899001,john.tan@mail.com
7 7,Siti Aisyah,MY778811,0187788112,aisyah@mail.com
8 8,Muhammad Amir,MY667722,0166677223,amir@mail.com
9 9,Emily Wong,MY556633,0125566334,emily@mail.com
10 10,Raj Kumar,MY445544,0194455445,raj@mail.com
11 11,Messi,342342,0783453,hasnain@gmail.,com
12 12,Messi ,TAE1920,+1245546445,messiiscool@gmail.com

```

## 2. Updating guest information

Requests the guest identification number. If the guest is found in guests.txt, the receptionist is prompted to key in the updated information. However, if the prompts are left blank, the information corresponding to that prompt does not change.

```

Press 1 to register a new guest
Press 2 to update guest information
Press 3 to view room availability
Press 4 to check-in guest
Press 5 to check-out guest
Press 6 to cancel booking
Press 7 to exit Receptionist menu
Enter your choice: 2
UPDATE GUEST INFORMATION
Enter Guest ID to update: 12
Enter new name (leave blank to keep current): Ronaldo
Enter new ID (leave blank to keep current): SIUU920
Enter new phone (leave blank to keep current): +90006789120
Enter new email (leave blank to keep current):
Guest information updated successfully.

```

```

attachments >  guests.txt
1,Ali Akber Manji,TP089001,0123456789,ali.manji@mail.com
2,Abdullah Shamim Naime,TP089002,0123456790,abdullah@mail.com
3,Dishil Atulkumar Rathod,TP089003,0123456791,dishil@mail.com
4,Jonathan Stephen Albest,TP089004,0123456792,jonathan@mail.com
5,Ahmed Ihab Ibrahim,TP089005,0123456793,ahmed@mail.com
6,John Tan,MY889900,0178899001,john.tan@mail.com
7,Siti Aisyah,MY778811,0187788112,aisyah@mail.com
8,Muhammad Amir,MY667722,0166677223,amir@mail.com
9,Emily Wong,MY556633,0125566334,emily@mail.com
10,Raj Kumar,MY445544,0194455445,raj@mail.com
11,Messi,342342,0783453,hasnain@gmail,.com
12,Ronaldo,SIUU920,+90006789120,messiiscool@gmail.com

```

### 3. Viewing room availability

Reads rooms.txt, proceeding to display the room ID, room type, price, and status.

F1STD-S1,STD-S,Single,1,109,AVAILABLE  
 F1STD-S2,STD-S,Single,1,109,OCCUPIED  
 F1STD-S3,STD-S,Single,1,109,AVAILABLE  
 F1STD-Q1,STD-Q,Queen,1,125,OCCUPIED  
 F1STD-Q2,STD-Q,Queen,1,125,AVAILABLE  
 F1STD-Q3,STD-Q,Queen,1,125,AVAILABLE  
 F1DLX-Q1,DLX-Q,Queen,1,159,OCCUPIED  
 F1DLX-Q2,DLX-Q,Queen,1,159,AVAILABLE  
 F1DLX-T1,DLX-T,Twin,1,167,OCCUPIED  
 F1DLX-31,DLX-3,Triple,1,189,AVAILABLE  
 F1FAM1,FAM,Family,1,200,OCCUPIED  
 F1FAM2,FAM,Family,1,200,AVAILABLE  
 F2STD-S1,STD-S,Single,2,109,AVAILABLE  
 F2STD-S2,STD-S,Single,2,109,OCCUPIED  
 F2STD-Q1,STD-Q,Queen,2,125,OCCUPIED  
 F2STD-Q2,STD-Q,Queen,2,125,AVAILABLE  
 F2DLX-Q1,DLX-Q,Queen,2,159,OCCUPIED  
 F2DLX-T1,DLX-T,Twin,2,167,AVAILABLE  
 F2FAM1,FAM,Family,2,200,AVAILABLE  
 F3STD-S1,STD-S,Single,3,109,AVAILABLE  
 F3STD-Q1,STD-Q,Queen,3,125,OCCUPIED  
 F3DLX-Q1,DLX-Q,Queen,3,159,AVAILABLE  
 F3DLX-T1,DLX-T,Twin,3,167,OCCUPIED  
 F3DLX-31,DLX-3,F4Triple,3,189,AVAILABLE  
 F3FAM1,FAM,Family,3,200,OCCUPIED  
 F4STD-S1,STD-S,Single,4,109,AVAILABLE  
 F4STD-Q1,STD-Q,Queen,4,125,AVAILABLE  
 F4DLX-Q1,DLX-Q,Queen,4,159,OCCUPIED  
 F4DLX-T1,DLX-T,Twin,4,167,AVAILABLE  
 F4FAM1,FAM,Family,4,200,AVAILABLE  
 F3STD-S67,STD-S,Single,3,109,AVAILABLE  
 F3FAM3,FAM,Family,3,200,AVAILABLE

```

Press 1 to register a new guest
Press 2 to update guest information
Press 3 to view room availability
Press 4 to check-in guest
Press 5 to check-out guest
Press 6 to cancel booking
Press 7 to exit Receptionist menu
Enter your choice: 3
VIEW ROOM AVAILABILITY
Room: F1STD-S1 | Type: STD-S | Price: 109 | Status: AVAILABLE
Room: F1STD-S2 | Type: STD-S | Price: 109 | Status: OCCUPIED
Room: F1STD-S3 | Type: STD-S | Price: 109 | Status: AVAILABLE
Room: F1STD-Q1 | Type: STD-Q | Price: 125 | Status: OCCUPIED
Room: F1STD-Q2 | Type: STD-Q | Price: 125 | Status: AVAILABLE
Room: F1STD-Q3 | Type: STD-Q | Price: 125 | Status: AVAILABLE
Room: F1DLX-Q1 | Type: DLX-Q | Price: 159 | Status: OCCUPIED
Room: F1DLX-Q2 | Type: DLX-Q | Price: 159 | Status: AVAILABLE
Room: F1DLX-T1 | Type: DLX-T | Price: 167 | Status: OCCUPIED
Room: F1DLX-31 | Type: DLX-3 | Price: 189 | Status: AVAILABLE
Room: F1FAM1 | Type: FAM | Price: 200 | Status: OCCUPIED
Room: F1FAM2 | Type: FAM | Price: 200 | Status: AVAILABLE
Room: F2STD-S1 | Type: STD-S | Price: 109 | Status: AVAILABLE
Room: F2STD-S2 | Type: STD-S | Price: 109 | Status: OCCUPIED
Room: F2STD-Q1 | Type: STD-Q | Price: 125 | Status: OCCUPIED
Room: F2STD-Q2 | Type: STD-Q | Price: 125 | Status: AVAILABLE
Room: F2DLX-Q1 | Type: DLX-Q | Price: 159 | Status: OCCUPIED
Room: F2DLX-T1 | Type: DLX-T | Price: 167 | Status: AVAILABLE
Room: F2FAM1 | Type: FAM | Price: 200 | Status: AVAILABLE
Room: F3STD-S1 | Type: STD-S | Price: 109 | Status: AVAILABLE
Room: F3STD-Q1 | Type: STD-Q | Price: 125 | Status: OCCUPIED
Room: F3DLX-Q1 | Type: DLX-Q | Price: 159 | Status: AVAILABLE
Room: F3DLX-T1 | Type: DLX-T | Price: 167 | Status: OCCUPIED
Room: F3DLX-31 | Type: DLX-3 | Price: 189 | Status: AVAILABLE
Room: F3FAM1 | Type: FAM | Price: 200 | Status: OCCUPIED
Room: F4STD-S1 | Type: STD-S | Price: 109 | Status: AVAILABLE
Room: F4STD-Q1 | Type: STD-Q | Price: 125 | Status: AVAILABLE
Room: F4DLX-Q1 | Type: DLX-Q | Price: 159 | Status: OCCUPIED
Room: F4DLX-T1 | Type: DLX-T | Price: 167 | Status: AVAILABLE
Room: F4FAM1 | Type: FAM | Price: 200 | Status: AVAILABLE
Room: F3STD-S67 | Type: STD-S | Price: 109 | Status: AVAILABLE
Room: F3FAM3 | Type: FAM | Price: 200 | Status: AVAILABLE

```

#### 4. Checking-in guests

Requests the receptionist to enter the guest identification number before reading through guests.txt to see if the guest exists. If the guest exists, the system displays all currently available rooms. Once the guest makes their choice, the receptionist enters the requested room ID. If the room ID is found in rooms.txt, its status is changed to OCCUPIED. The receptionist is then prompted to key in the check-in date and the expected check-out date. A booking identification number, assigned similarly to when registering guests, along with the guest identification number, selected room ID, check-in date, check-out date, and the status CHECKED\_IN, is appended to bookings.txt.

```

Press 1 to register a new guest
Press 2 to update guest information
Press 3 to view room availability
Press 4 to check-in guest
Press 5 to check-out guest
Press 6 to cancel booking
Press 7 to exit Receptionist menu
Enter your choice: 4
CHECK-IN GUEST
Enter Guest ID: 12
Available rooms:
Room: F1STD-S1 | Type: STD-S | Price: 109
Room: F1STD-S3 | Type: STD-S | Price: 109
Room: F1STD-Q2 | Type: STD-Q | Price: 125
Room: F1STD-Q3 | Type: STD-Q | Price: 125
Room: F1DLX-Q2 | Type: DLX-Q | Price: 159
Room: F1DLX-31 | Type: DLX-3 | Price: 189
Room: F1FAM2 | Type: FAM | Price: 200
Room: F2STD-S1 | Type: STD-S | Price: 109
Room: F2STD-Q2 | Type: STD-Q | Price: 125
Room: F2DLX-T1 | Type: DLX-T | Price: 167
Room: F2FAM1 | Type: FAM | Price: 200
Room: F3STD-S1 | Type: STD-S | Price: 109
Room: F3DLX-Q1 | Type: DLX-Q | Price: 159
Room: F3DLX-31 | Type: DLX-3 | Price: 189
Room: F4STD-S1 | Type: STD-S | Price: 109
Room: F4STD-Q1 | Type: STD-Q | Price: 125
Room: F4DLX-T1 | Type: DLX-T | Price: 167
Room: F4FAM1 | Type: FAM | Price: 200
Room: F3STD-S67 | Type: STD-S | Price: 109
Room: F3FAM3 | Type: FAM | Price: 200
Enter Room ID to assign: F3FAM3
Enter check-in date (DD-MM-YYYY): 05-12-2006
Enter expected check-out date (DD-MM-YYYY): 24-12-2006
Guest checked in successfully with Booking ID: 2011

```

**F3FAM3,FAM,Family,3,200,OCCUPIED**

chments >  bookings.txt

```

2001,1,F1STD-S2,2025-12-20,2025-12-22,CHECKED_OUT
2002,2,F1DLX-Q1,2025-12-22,2025-12-26,CHECKED_IN
2003,3,F1FAM1,2025-12-21,2025-12-25,CHECKED_IN
2004,4,F2STD-S2,2025-12-23,2025-12-24,CHECKED_OUT
2005,5,F2DLX-Q1,2025-12-24,2025-12-27,CHECKED_IN
2006,6,F3STD-Q1,2025-12-20,2025-12-23,CHECKED_OUT
2007,7,F3DLX-T1,2025-12-22,2025-12-26,CHECKED_IN
2008,8,F4DLX-Q1,2025-12-24,2025-12-28,CHECKED_IN
2009,9,F2DLX-32,2025-12-23,2025-12-25,CHECKED_OUT
2010,10,F3FAM1,2025-12-24,2025-12-29,CHECKED_IN
2011,12,F3FAM3,05-12-2006,24-12-2006,CHECKED_IN

```

## 5. Checking-out guests

Requests the receptionist to key in the booking identification number before checking whether it exists inside bookings.txt. If the booking identification exists, the booking's status

is changed to CHECKED\_OUT in bookings.txt, while the checked-out room's status is changed to AVAILABLE in rooms.txt.

```

Press 1 to register a new guest
Press 2 to update guest information
Press 3 to view room availability
Press 4 to check-in guest
Press 5 to check-out guest
Press 6 to cancel booking
Press 7 to exit Receptionist menu
Enter your choice: 5
CHECK-OUT GUEST
Enter Booking ID: 2011
Guest checked out successfully. Direct guest to Accountant for billing.

```

**F3FAM3, FAM, Family, 3, 200, AVAILABLE**

**2011, 12, F3FAM3, 05-12-2006, 24-12-2006, CHECKED\_OUT**

## 6. Cancelling bookings

Requests the receptionist to enter the booking identification number before verifying that it exists inside bookings.txt. If the booking identification exists, the booking's status is checked to see if it CHECKED\_IN. If the status is CHECKED\_IN, the system changes the status to CANCELLED in bookings.txt, while the cancelled room's status is changed to AVAILABLE in rooms.txt.

```

Press 1 to register a new guest
Press 2 to update guest information
Press 3 to view room availability
Press 4 to check-in guest
Press 5 to check-out guest
Press 6 to cancel booking
Press 7 to exit Receptionist menu
Enter your choice: 6
CANCEL BOOKING
Enter Booking ID to cancel: 2010
Booking cancelled successfully.

```

chments >  bookings.txt

```
2001,1,F1STD-S2,2025-12-20,2025-12-22,CHECKED_OUT
2002,2,F1DLX-Q1,2025-12-22,2025-12-26,CHECKED_IN
2003,3,F1FAM1,2025-12-21,2025-12-25,CHECKED_IN
2004,4,F2STD-S2,2025-12-23,2025-12-24,CHECKED_OUT
2005,5,F2DLX-Q1,2025-12-24,2025-12-27,CHECKED_IN
2006,6,F3STD-Q1,2025-12-20,2025-12-23,CHECKED_OUT
2007,7,F3DLX-T1,2025-12-22,2025-12-26,CHECKED_IN
2008,8,F4DLX-Q1,2025-12-24,2025-12-28,CHECKED_IN
2009,9,F2DLX-32,2025-12-23,2025-12-25,CHECKED_OUT
2010,10,F3FAM1,2025-12-24,2025-12-29,CANCELLED
2011,12,F3FAM3,05-12-2006,24-12-2006,CHECKED_OUT
```

**F3FAM1,FAM,Family,3,200,AVAILABLE**

## 7. Exiting receptionist module

Sends the receptionist back to the main menu.

```

Press 1 to register a new guest
Press 2 to update guest information
Press 3 to view room availability
Press 4 to check-in guest
Press 5 to check-out guest
Press 6 to cancel booking
Press 7 to exit Receptionist menu
Enter your choice: 7
Exiting Receptionist menu.
```

## 4.4 Accountant

As shown below, the accountant's system shows 4 main functions: Recording a payment, generating an income report, viewing any outstanding payments and generating a monthly financial summary. The last function exits the accountant system and goes back to the main menu.

```
=== ACCOUNTING SYSTEM ===
1. Record Payment
2. Generate Income Report
3. View Outstanding Payments
4. Generate Monthly Financial Summary
5. Exit Accounting System
Enter your choice:
```

### 1) Record Payment

Asks the accountant to enter the details of the booking ID, payment amount, whether it has been paid or not and the date. These details are then written and updated into the 'payments.txt' and 'daily\_logs.txt' files.

```

=== ACCOUNTING SYSTEM ===
1. Record Payment
2. Generate Income Report
3. View Outstanding Payments
4. Generate Monthly Financial Summary
5. Exit Accounting System
Enter your choice: 1

=== RECORD PAYMENT ===
Enter Booking ID: 2001
Enter Payment Amount: 200
Enter Payment Status (Paid/Unpaid): paid
Enter Payment Date (YYYY-MM-DD): 2025-11-23
Payment successfully recorded.

```

```

2001,218,2025-12-22,Paid
2002,636,2025-12-22,Unpaid
2003,800,2025-12-23,Paid
2004,109,2025-12-24,Paid
2005,477,2025-12-24,Unpaid
2006,375,2025-12-23,Paid
2007,668,2025-12-22,Unpaid
2008,636,2025-12-24,Unpaid
2009,378,2025-12-25,Paid
2010,1000,2025-12-24,Unpaid
2001,200,2025-12-22,Paid
2001,200,2025-11-23,Paid

```

## 2) Generate Income Report

Calculates the total sum of income from payments file.

```
=== ACCOUNTING SYSTEM ===
1. Record Payment
2. Generate Income Report
3. View Outstanding Payments
4. Generate Monthly Financial Summary
5. Exit Accounting System
Enter your choice: 2

=== GENERATING INCOME REPORT ===
Total Income Collected: RM 2280
```

### 3) View Outstanding Payments

Shows details of all bookings that are unpaid.

```
5. Exit Accounting System
Enter your choice: 3

=== OUTSTANDING PAYMENTS ===
Booking ID: 2002
Amount Due: RM 636
Date Recorded: 2025-12-22

Booking ID: 2005
Amount Due: RM 477
Date Recorded: 2025-12-24

Booking ID: 2007
Amount Due: RM 668
Date Recorded: 2025-12-22

Booking ID: 2008
Amount Due: RM 636
Date Recorded: 2025-12-24

Booking ID: 2010
Amount Due: RM 1000
Date Recorded: 2025-12-24

```

#### 4) Generate Monthly Financial Summary

Asks user to enter the month needed, shows a separated total of paid and unpaid in that month.

```
=== ACCOUNTING SYSTEM ===
1. Record Payment
2. Generate Income Report
3. View Outstanding Payments
4. Generate Monthly Financial Summary
5. Exit Accounting System
Enter your choice: 4

=== MONTHLY FINANCIAL SUMMARY ===
Enter Month (YYYY-MM): 2025-12
Monthly Summary for 2025-12
Total Paid: RM 2080
Total Unpaid: RM 3417
```

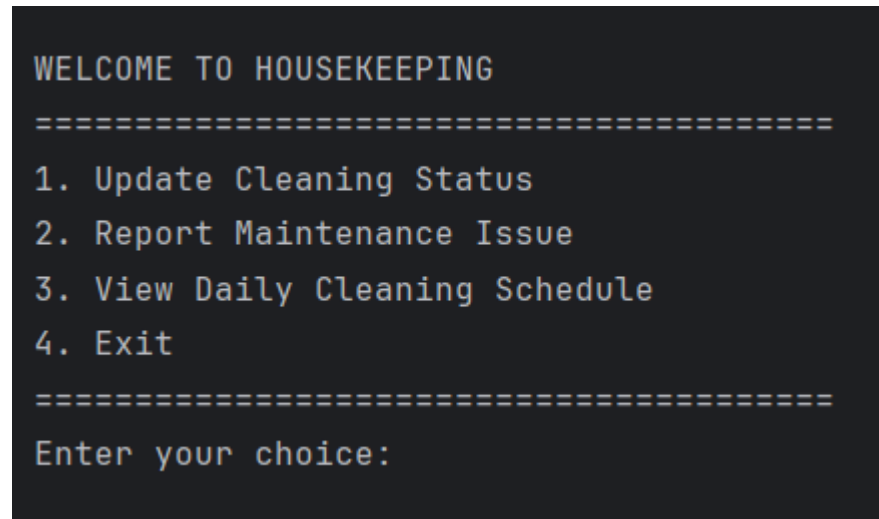
### 5) Exit Accounting System

Sends the accountant back to the main menu

```
=== ACCOUNTING SYSTEM ===
1. Record Payment
2. Generate Income Report
3. View Outstanding Payments
4. Generate Monthly Financial Summary
5. Exit Accounting System
Enter your choice: 5
Exiting Accounting System...
```

#### 4.5 Housekeeping

Below, the housekeeping is allowed 3 operations excluding the exit choice, which are Updating Cleaning Status, Report Maintenance Issue and View Daily Cleaning Status.



```
WELCOME TO HOUSEKEEPING
=====
1. Update Cleaning Status
2. Report Maintenance Issue
3. View Daily Cleaning Schedule
4. Exit
=====
Enter your choice:
```

##### 1) Update Cleaning Status

The screenshot below shows the user choosing option 1 - the 'Update Cleaning Status' option. The system asks the user to enter the room number to change status, to which the user responded with 'F1STD-S1'. After system validates the input and updates the records, the user is prompt to enter the date of today in the format (YYYY-MM-DD) and the comment of the operation to send to the cleaning log. After the system validates inputs, the inputs are logged successfully, and a message is informed to the user of this action.

```

WELCOME TO HOUSEKEEPING
=====
1. Update Cleaning Status
2. Report Maintenance Issue
3. View Daily Cleaning Schedule
4. Exit
=====
Enter your choice: 1
Enter the room number: F1STD-S1
Room F1STD-S1 cleaning status was changed to Clean
Enter the date of today (yyyy-MM-dd): 2026-01-01
Enter the comment: Room Cleaned successfully
Cleaning status was changed for F1STD-S1

```



The screenshot shows a code editor with two tabs: 'Housekeeping.py' and 'Room\_Cleaning\_Status.txt'. The 'Room\_Cleaning\_Status.txt' tab is active, displaying a list of room IDs and their cleaning status (True or False). The list is as follows:

|    |                 |
|----|-----------------|
| 1  | F1STD-S1, True  |
| 2  | F1STD-S2, False |
| 3  | F1STD-S3, False |
| 4  | F1STD-Q1, False |
| 5  | F1STD-Q2, False |
| 6  | F1STD-Q3, False |
| 7  | F1DLX-Q1, False |
| 8  | F1DLX-Q2, False |
| 9  | F1DLX-T1, False |
| 10 | F1DLX-31, False |
| 11 | F1FAM1, False   |
| 12 | F1FAM2, False   |
| 13 | F2STD-S1, False |
| 14 | F2STD-S2, False |

```

Housekeeping.py x Cleaning_Log.txt x Room_Clean
1 2026-01-06,F1STD-Q2,Room Clean and Tidy
2 2026-01-06,F1STD-Q3,Room Cleaned
3 2026-01-06,F4FAM1,Cleaned
4 2026-01-07,F3STD-S1,Room Cleaned
5 2026-01-07,F1STD-S1,Room Cleaned Successfully
6

```

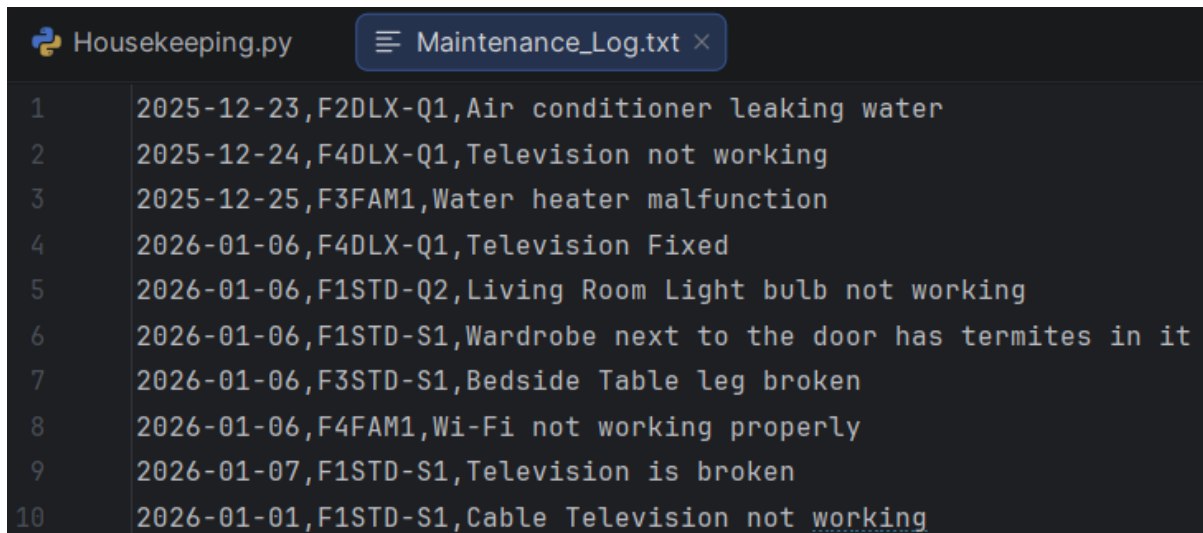
## 2) Report Maintenance Issue

The screenshot below shows the user choosing option 2 - the 'Report Maintenance Issue' option. The system asks the user to enter today's date, the room number and the maintenance issue as a comment. After the system validates the input and records the report, the system displays a successful message.

```

WELCOME TO HOUSEKEEPING
=====
1. Update Cleaning Status
2. Report Maintenance Issue
3. View Daily Cleaning Schedule
4. Exit
=====
Enter your choice: 2
Enter the date of today (yyyy-MM-dd): 2026-01-01
Enter the room number: F1STD-S1
What seems to be the issue?
Cable Television not working
Maintenance issue was successfully reported

```



```
Housekeeping.py Maintenance_Log.txt x
1 2025-12-23,F2DLX-Q1,Air conditioner leaking water
2 2025-12-24,F4DLX-Q1,Television not working
3 2025-12-25,F3FAM1,Water heater malfunction
4 2026-01-06,F4DLX-Q1,Television Fixed
5 2026-01-06,F1STD-Q2,Living Room Light bulb not working
6 2026-01-06,F1STD-S1,Wardrobe next to the door has termites in it
7 2026-01-06,F3STD-S1,Bedside Table leg broken
8 2026-01-06,F4FAM1,Wi-Fi not working properly
9 2026-01-07,F1STD-S1,Television is broken
10 2026-01-01,F1STD-S1,Cable Television not working
```

### 3) View Daily Cleaning Schedule

The screenshot below shows the user choosing option 3 - the 'View Daily Cleaning Schedule' option. The system Fetches the rooms and its cleaning status, separating the rooms which are not cleaned from the rooms which are cleaned. Since the user already set the status of room F1STD-S1 to cleaned, it is marked as Cleaned, and the rest are uncleaned.

```

WELCOME TO HOUSEKEEPING
=====
1. Update Cleaning Status
2. Report Maintenance Issue
3. View Daily Cleaning Schedule
4. Exit
=====
Enter your choice: 3
Room: F1STD-S2 Cleaning Status: Not Cleaned
Room: F1STD-S3 Cleaning Status: Not Cleaned
Room: F1STD-Q1 Cleaning Status: Not Cleaned
Room: F1STD-Q2 Cleaning Status: Not Cleaned
Room: F1STD-Q3 Cleaning Status: Not Cleaned
Room: F1DLX-Q1 Cleaning Status: Not Cleaned
Room: F1DLX-Q2 Cleaning Status: Not Cleaned
Room: F1DLX-T1 Cleaning Status: Not Cleaned
Room: F1DLX-31 Cleaning Status: Not Cleaned
Room: F1FAM1 Cleaning Status: Not Cleaned
Room: F1FAM2 Cleaning Status: Not Cleaned
Room: F2STD-S1 Cleaning Status: Not Cleaned
Room: F2STD-S2 Cleaning Status: Not Cleaned
Room: F2STD-Q1 Cleaning Status: Not Cleaned
Room: F2STD-Q2 Cleaning Status: Not Cleaned
Room: F2DLX-Q1 Cleaning Status: Not Cleaned
Room: F2DLX-T1 Cleaning Status: Not Cleaned
Room: F2DLX-32 Cleaning Status: Not Cleaned
Room: F2FAM1 Cleaning Status: Not Cleaned
Room: F3STD-S1 Cleaning Status: Not Cleaned
Room: F3STD-Q1 Cleaning Status: Not Cleaned
Room: F3DLX-Q1 Cleaning Status: Not Cleaned
Room: F3DLX-T1 Cleaning Status: Not Cleaned
Room: F3DLX-31 Cleaning Status: Not Cleaned

Room: F3FAM1 Cleaning Status: Not Cleaned
Room: F4STD-S1 Cleaning Status: Not Cleaned
Room: F4STD-Q1 Cleaning Status: Not Cleaned
Room: F4DLX-Q1 Cleaning Status: Not Cleaned
Room: F4DLX-T1 Cleaning Status: Not Cleaned
Room: F4FAM1 Cleaning Status: Not Cleaned

Room: F1STD-S1 Cleaning Status: Cleaned

```

#### 4.6 Guest

Figure 4.6.1: Guest menu displays available options

```

=====

Welcome to The Guest Menu
1. View available rooms
2. Make a reservation
3. Cancel a reservation
4. View billing summary
5. Exit
Enter your choice:

```

Figure 4.6.2: Display of available rooms, when making a reservation. The user is prompted to choose and enter a room ID

```

Enter your choice: 2

Make a reservation
Available rooms
| Room ID: F1STD-S1 | Occupancy: Single | Price: 109
| Room ID: F1STD-S2 | Occupancy: Single | Price: 109
| Room ID: F1STD-S3 | Occupancy: Single | Price: 109
| Room ID: F1STD-Q2 | Occupancy: Queen | Price: 125
| Room ID: F1STD-Q3 | Occupancy: Queen | Price: 125
| Room ID: F1DLX-Q1 | Occupancy: Queen | Price: 159
| Room ID: F1DLX-Q2 | Occupancy: Queen | Price: 159
| Room ID: F1DLX-31 | Occupancy: Triple | Price: 189
| Room ID: F1FAM1 | Occupancy: Family | Price: 200
| Room ID: F1FAM2 | Occupancy: Family | Price: 200
| Room ID: F2STD-S1 | Occupancy: Single | Price: 109
| Room ID: F2STD-Q2 | Occupancy: Queen | Price: 125
| Room ID: F2DLX-T1 | Occupancy: Twin | Price: 167
| Room ID: F2FAM1 | Occupancy: Family | Price: 200
| Room ID: F3STD-S1 | Occupancy: Single | Price: 109
| Room ID: F3DLX-Q1 | Occupancy: Queen | Price: 159
| Room ID: F3DLX-31 | Occupancy: Triple | Price: 189
| Room ID: F4STD-S1 | Occupancy: Single | Price: 109
| Room ID: F4STD-Q1 | Occupancy: Queen | Price: 125
| Room ID: F4DLX-T1 | Occupancy: Twin | Price: 167
| Room ID: F4FAM1 | Occupancy: Family | Price: 200

To reserve, please enter your room ID:

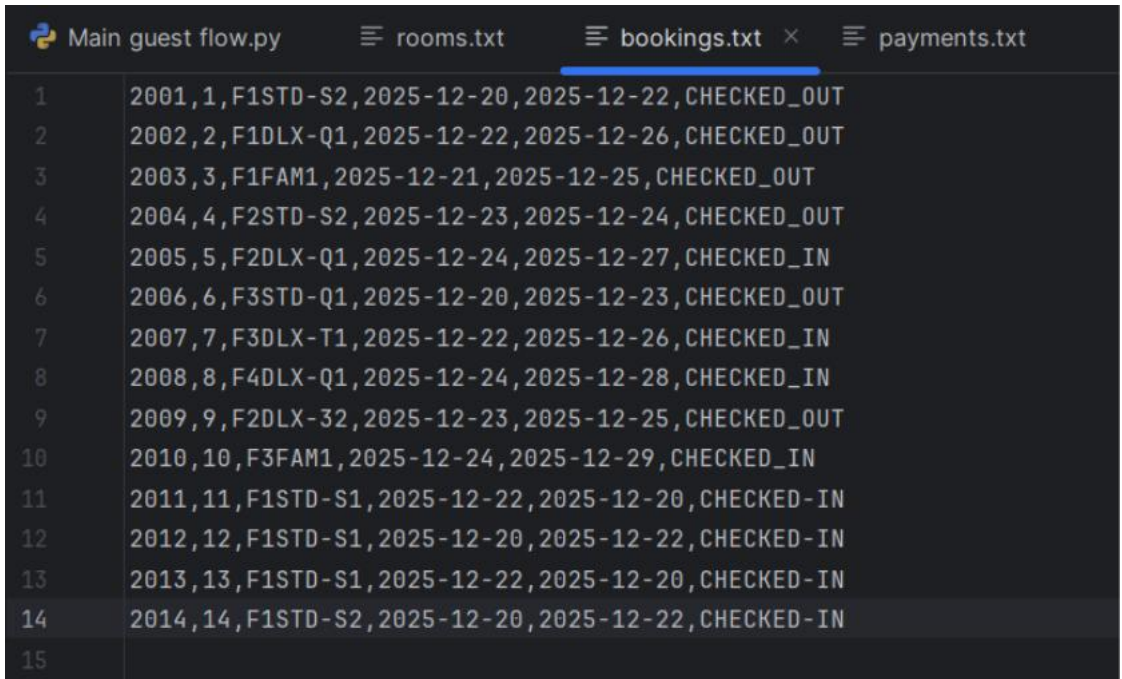
```

Figure 4.6.3: The user is prompted to enter a roomID, the respective dates for checking and checked out, to make a reservation. Which finally then gives a booking ID.

```

To reserve, please enter your room ID: F1STD-S2
You selected: F1STD-S2
Enter check-in date (YYYY-MM-DD): 2025-12-20
Enter check-out date (YYYY-MM-DD): 2025-12-22
You have successfully booked a reservation, your Booking ID: 2014

```



| Booking ID | Room ID  | Check-in Date | Check-out Date | Status      |
|------------|----------|---------------|----------------|-------------|
| 2001       | F1STD-S2 | 2025-12-20    | 2025-12-22     | CHECKED_OUT |
| 2002       | F1DLX-Q1 | 2025-12-22    | 2025-12-26     | CHECKED_OUT |
| 2003       | F1FAM1   | 2025-12-21    | 2025-12-25     | CHECKED_OUT |
| 2004       | F2STD-S2 | 2025-12-23    | 2025-12-24     | CHECKED_OUT |
| 2005       | F2DLX-Q1 | 2025-12-24    | 2025-12-27     | CHECKED_IN  |
| 2006       | F3STD-Q1 | 2025-12-20    | 2025-12-23     | CHECKED_OUT |
| 2007       | F3DLX-T1 | 2025-12-22    | 2025-12-26     | CHECKED_IN  |
| 2008       | F4DLX-Q1 | 2025-12-24    | 2025-12-28     | CHECKED_IN  |
| 2009       | F2DLX-32 | 2025-12-23    | 2025-12-25     | CHECKED_OUT |
| 2010       | F3FAM1   | 2025-12-24    | 2025-12-29     | CHECKED_IN  |
| 2011       | F1STD-S1 | 2025-12-22    | 2025-12-20     | CHECKED_IN  |
| 2012       | F1STD-S1 | 2025-12-20    | 2025-12-22     | CHECKED_IN  |
| 2013       | F1STD-S1 | 2025-12-22    | 2025-12-20     | CHECKED_IN  |
| 2014       | F1STD-S2 | 2025-12-20    | 2025-12-22     | CHECKED_IN  |

Figure 4.6.4: User can cancel the reservation. The cancellation screenshot continues with the previously used booking ID: 2014, which was just used to make a reservation, so these further demonstrations show the cancellation process.

```

Enter your choice: 3

Cancel reservation(s)
Enter your booking ID: 2014
Booking ID is 2014
Successfully canceled reservation

```

Figure 4.6.4.1: Shows the updated status to “CHECKED\_OUT”

```

14 2014,14,F1STD-S2,2025-12-20,2025-12-22,CHECKED_OUT
15

```

Figure 4.6.5: Viewing billing summary shows the relevant information without having to ask the user for sensitive information like payment ID. Instead, it asks for booking ID.

```

Enter your choice: 4

View billing summary
=====
Enter your Booking ID: 2001
Amount: 218 | Date: 2025-12-22 | Status: Paid

```

Figure 4.6.6: Exits the code

```

Enter your choice: 5

Exiting Guest Menu

Process finished with exit code 0

```

## 5.0 Conclusion

### 5.1 Advantages of the System

#### 1) Role Selection

This is seen on MAIN\_PAGE where users pick their respective roles which call their corresponding functions to perform their assigned tasks efficiently, while maintaining StayEase's system organisation, data integrity, and separate responsibilities without affecting other parts of the system.

#### 2) Menu-driven Interface

The simple and user-friendly menu-driven interface allows for smooth navigation for all roles, limiting user mis-inputs and simplifying system usage, while allowing for quick and efficient staff training in the future.

#### 3) Uses CSV-based Text Files

CSV-formatted text file storage provides structured, readable, and adaptable data management without relying on a database. This makes records easier to read, write, update, and validate during file handling operations.

#### **4) Automated Handling of Core Operations**

The automated handling of operations such as bookings, payments, and room status improves the overall operational efficiency by reducing manual labour through automating repetitive tasks such as booking updates, minimising the need for manual cross-checking files, and limiting the occurrence of human errors across all roles.

#### **5) Data Storage as Program Runs**

Data is stored throughout different modules, ensuring all records remain saved and accessible system-wide for multiple program executions, maintaining system-wide consistency.

#### **6) Modular Programming**

Modular programming breaks down the complex StayEase system into smaller, and simpler parts, improving bug identification and resolution, and updating system components without affecting others.

### **5.2 Limitations of the developed system**

#### **1) Duplicate data Or Cross Checking:**

There is a strict way of detecting any same duplicated data formed for example the manger can write the same room accidentally or the receptionist can only check the room status and assign it but does not verify the calendar availability against existing booking:

- ❖ The room can be set to “AVAILABLE” by the manger without referring to the booking files as it has no function no cross reference both files of room occupancy and the booking records.

#### **2) Single User Only**

Only one user can use one at a time as it can't handle multiple users at the same time if 2 staff members try to edit any of the files simultaneously will end up overwriting each other updated files thus a major flaw.

### **3) Input Verification**

The system also assumes that all the input put are all valid and correctly formatted but there is no way of validating for example register with number with words instead of actual numbers and the file will end up saving these without any proper verification of it thus the integrity of the data is extremely dependent on the user discipline. Therefore, resulting in inconsistent results.

### **4) User Authorization**

While switching roles are fast, the system fails to authorize users to access roles. For example, a Guest – who is only allowed to make reservations, can access the Manager's Menu, giving them the control of the System's Admin privileges. This makes the system less protected, because anyone can access roles easily, with no password system.

### **5) Data Lookup Scalability Limitation**

Though initially the format of data is well recorded, it will soon be vulnerable to slowness in system lookup. When thousands of data is entered, the system's lookup functions operating these large datasets may slow down the system, moreover, if it is possible that there is not enough memory to read these large datasets, causing the system to slow significantly and, in extreme cases, risk memory exhaustion.

## **5.3 Suggestions for Future Improvements and System Enhancements**

The system currently uses text files for data storage, while this approach is simple and suitable for small-scale usage, using a database in the future would improve data security, reduce redundancy, and allow faster data retrieval. A database would also support shared access of multiple users more efficiently.

Another possible improvement could be the security in the role-based access control. Even though the system separates responsibilities by role, it lacks security measures such as user

authentication and passwords, which could be used to prevent unauthorised access to sensitive information, like financial data, and guest's contact information.

Improving the user interface would help to have a user-friendly experience that is interactive and easy for the user to remember important pieces of information. Consistent design and clear navigation enable the user to complete tasks quickly which then further reduces the chances of running into an error.

Implementing a guest login system where everything is automatically set to the user's information after inputted. Instead of having to manually input Guest IDs. For example, adding a "Remember me" function can speed up the process quickly that displays all relevant information just after clicking one of the main menu options. This would reduce the code necessary to be repeatedly typed.

## 6.0 References

MySoftinn. (n.d.). A comprehensive step-by-step guide to e-invoicing for Malaysian hotels. <https://page.mysoftinn.com/en/a-comprehensive-step-by-step-guide-to-e-invoicing-for-malaysian-hotels>

SiteMinder. (2022, October 12). A full guide to hotel departments and their functions.

SiteMinder. <https://www.siteminder.com/r/hotel-departments-building-functions/>

The Edge Malaysia. (2023, December 18). E-invoicing rollout in Malaysia to start in phases from August 2024. <https://theedgemalaysia.com/node/728416>

W3Schools. (2019). *Python File Write*. W3schools.com.

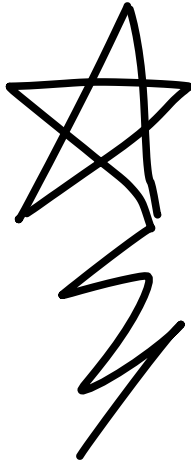
[https://www.w3schools.com/python/python\\_file\\_write.asp](https://www.w3schools.com/python/python_file_write.asp)

## 7.0 Appendix

### 7.1 Workload Matrix

| Student name | TP number | Contribution | Signature |
|--------------|-----------|--------------|-----------|
|--------------|-----------|--------------|-----------|

|                                               |          |                                                                                                                                                                                                                                                                                                             |                                                                                      |
|-----------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Abdullah Shamim<br>Naime<br>(GROUP<br>LEADER) | TP100781 | <ul style="list-style-type: none"> <li>-Layout of the Document</li> <li>-Introduction</li> <li>-Business Rules</li> <li>-Manager Assumption</li> <li>-Main Menu Structure</li> <li>-System Design of Manager</li> <li>- part of limitations of the developed system</li> </ul>                              |    |
| Dishil AtulKumar<br>Rathod                    | TP091840 | <ul style="list-style-type: none"> <li>-Accountant pseudocode</li> <li>-Concepts explanation</li> <li>-Screenshots of program execution</li> <li>- Accountant Assumptions</li> <li>-System design for Accountant</li> <li>-First two suggestions for future improvements and system enhancements</li> </ul> |   |
| Jonathan Stephen<br>Albest                    | TP090512 | <ul style="list-style-type: none"> <li>- Housekeeping assumptions.</li> <li>- Housekeeping pseudocode.</li> <li>- Housekeeping Explanation of</li> </ul>                                                                                                                                                    |  |

|                 |          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                     |
|-----------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|                 |          | <p>programming</p> <p>Concepts Applied</p> <ul style="list-style-type: none"> <li>- Housekeeping</li> </ul> <p>screenshots of</p> <p>program execution</p> <ul style="list-style-type: none"> <li>- Part of limitations</li> </ul> <p>of the developed</p> <p>system</p> <ul style="list-style-type: none"> <li>- Creation of</li> </ul> <p>cleaning_log,</p> <p>maintenance_log, and</p> <p>room_cleaning_status</p>                                                                                                                        |                                                                                     |
| Ali Akber Manji | TP100852 | <p>-Proofreading</p> <p>(formatting,</p> <p>consistency, and</p> <p>typo-level</p> <p>corrections)</p> <p>-Receptionist</p> <p>assumption,</p> <p>pseudocode, code,</p> <p>programming</p> <p>concept explanation,</p> <p>and screenshot of</p> <p>program execution</p> <ul style="list-style-type: none"> <li>- creation of text files</li> </ul> <p>excluding</p> <p>cleaning_Log,</p> <p>maintenance_Log,</p> <p>and</p> <p>room_cleaning_status</p> <ul style="list-style-type: none"> <li>- Advantages of the</li> </ul> <p>system</p> |  |

|                       |          |                                                                                                            |                                                                                    |
|-----------------------|----------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
|                       |          | -Part of layout                                                                                            |                                                                                    |
| Ahmed Ihab<br>Ibrahim | TP116875 | -Guest assumption<br>-Pseudocode, code<br>for guest<br>-Parts of suggestions<br>for future<br>improvements |  |

## 7.2 Screenshot of Text Files Contents

### Bookings.txt

```
bookings.txt
2001,1,F1STD-S2,2025-12-20,2025-12-22,CHECKED_OUT
2002,2,F1DLX-Q1,2025-12-22,2025-12-26,CHECKED_IN
2003,3,F1FAM1,2025-12-21,2025-12-25,CHECKED_IN
2004,4,F2STD-S2,2025-12-23,2025-12-24,CHECKED_OUT
2005,5,F2DLX-Q1,2025-12-24,2025-12-27,CHECKED_IN
2006,6,F3STD-Q1,2025-12-20,2025-12-23,CHECKED_OUT
2007,7,F3DLX-T1,2025-12-22,2025-12-26,CHECKED_IN
2008,8,F4DLX-Q1,2025-12-24,2025-12-28,CHECKED_IN
2009,9,F2DLX-32,2025-12-23,2025-12-25,CHECKED_OUT
2010,10,F3FAM1,2025-12-24,2025-12-29,CHECKED_IN
2011,11,F4FAM1,05-06-2026,10-06-2026,CANCELLED
```

### Cleaning\_Log.txt

```
Cleaning_Log.txt
2026-01-07,F1STD-S1,Room Cleaned successfully
2026-01-07,F1STD-S1,Cleaned Room
2026-01-07,F1STD-S1,Cleaned Room
2026-01-07,F1STD-S1,Room Cleaned
2026-01-07,F1STD-S2,Room Cleaned
2026-01-08,F1STD-S1,Room was cleaned successfully
2026-01-09,F7FAM1,Cleaned
2026-01-09,F5FAM1,Room Has been Cleaned
2026-01-09,F5FAM1,Room was cleaned
```

### daily\_logs.txt

```
daily_logs.txt
2025-12-22,BOOKING,2002,Guest 2 checked in to F1DLX-Q1
2025-12-22,PAYMENT,2001,Payment RM 218 received
2025-12-23,BOOKING,2003,Family room occupied on floor 1
2025-12-24,MAINTENANCE,F4DLX-Q1,TV reported faulty
2025-12-24,PAYMENT,2004,Payment RM 109 received
2025-12-25,CLEANING,F3FAM1,Cleaning pending due to late checkout
```

## Guests.txt

```

● ● ● ● guests.txt
1,Ali Akber Manji,TP089001,0123456789,ali.manji@mail.com
2,Abdullah Shamim Naime,TP089002,0123456790,abdullah@mail.com
3,Dishil Atulkumar Rathod,TP089003,0123456791,dishil@mail.com
4,Jonathan Stephen Albest,TP089004,0123456792,jonathan@mail.com
5,Ahmed Ihab Ibrahim,TP089005,0123456793,ahmed@mail.com
6,John Tan,MY889900,0178899001,john.tan@mail.com
7,Siti Aisyah,MY778811,0187788112,aisyah@mail.com
8,Muhammad Amir,MY667722,0166677223,amir@mail.com
9,Emily Wong,MY556633,0125566334,emily@mail.com
10,Raj Kumar,MY445544,0194455445,raj@mail.com
11,Messi,ARG007,0783456098,messibusiness@gmail.com

```

## Maintenance\_Log.txt

```

● ● ● ● Maintenance_Log.txt
2025-12-23,F2DLX-Q1,Air conditioner leaking water
2025-12-24,F4DLX-Q1,Television not working
2025-12-25,F3FAM1,Water heater malfunction
2026-01-06,F4DLX-Q1,Television Fixed
2026-01-06,F1STD-Q2,Living Room Light bulb not working
2026-01-06,F1STD-S1,Wardrobe next to the door has termites in it
2026-01-06,F3STD-S1,Bedside Table leg broken
2026-01-06,F4FAM1,Wi-Fi not working properly

```

## Payments.txt

```

● ● ● ● payments.txt
2001,218,2025-12-22,Paid
2002,636,2025-12-22,Unpaid
2003,800,2025-12-23,Paid
2004,109,2025-12-24,Paid
2005,477,2025-12-24,Unpaid
2006,375,2025-12-23,Paid
2007,668,2025-12-22,Unpaid
2008,636,2025-12-24,Unpaid
2009,378,2025-12-25,Paid
2010,1000,2025-12-24,Unpaid

```

## Room\_Cleaning\_Status.txt

 Room\_Cleaning\_Status.txt

```

F1STD-S1,False
F1STD-S2,False
F1STD-S3,False
F1STD-Q1,False
F1STD-Q2,False
F1STD-Q3,False
F1DLX-Q1,False
F1DLX-Q2,False
F1DLX-T1,False
F1DLX-31,False
F1FAM1,False
F1FAM2,False
F2STD-S1,False
F2STD-S2,False
F2STD-Q1,False
F2STD-Q2,False
F2DLX-Q1,False
F2DLX-T1,False
F2DLX-32,False
F2FAM1,False
F3STD-S1,False
F3STD-Q1,False
F3DLX-Q1,False
F3DLX-T1,False
F3DLX-31,False
F3FAM1,False
F4STD-S1,False
F4STD-Q1,False
F4DLX-Q1,False
F4DLX-T1,False
F4FAM1,False

```

## Rooms.txt

 rooms.txt

```

F1STD-S1,STD-S,Single,1,109,AVAILABLE
F1STD-S2,STD-S,Single,1,109,OCCUPIED
F1STD-S3,STD-S,Single,1,109,AVAILABLE
F1STD-Q1,STD-Q,Queen,1,125,OCCUPIED
F1STD-Q2,STD-Q,Queen,1,125,AVAILABLE
F1STD-Q3,STD-Q,Queen,1,125,AVAILABLE
F1DLX-Q1,DLX-Q,Queen,1,159,OCCUPIED
F1DLX-Q2,DLX-Q,Queen,1,159,AVAILABLE
F1DLX-T1,DLX-T,Twin,1,167,OCCUPIED
F1DLX-31,DLX-3,Triple,1,189,AVAILABLE
F1FAM1,FAM,Family,1,200,OCCUPIED
F1FAM2,FAM,Family,1,200,AVAILABLE
F2STD-S1,STD-S,Single,2,109,AVAILABLE
F2STD-S2,STD-S,Single,2,109,OCCUPIED
F2STD-Q1,STD-Q,Queen,2,125,OCCUPIED
F2STD-Q2,STD-Q,Queen,2,125,AVAILABLE
F2DLX-Q1,DLX-Q,Queen,2,159,OCCUPIED
F2DLX-T1,DLX-T,Twin,2,167,AVAILABLE
F2DLX-32,DLX-3,Triple,2,189,OCCUPIED
F2FAM1,FAM,Family,2,200,AVAILABLE
F3STD-S1,STD-S,Single,3,109,AVAILABLE
F3STD-Q1,STD-Q,Queen,3,125,OCCUPIED
F3DLX-Q1,DLX-Q,Queen,3,159,AVAILABLE
F3DLX-T1,DLX-T,Twin,3,167,OCCUPIED
F3DLX-31,DLX-3,Triple,3,189,AVAILABLE
F3FAM1,FAM,Family,3,200,OCCUPIED
F4STD-S1,STD-S,Single,4,109,AVAILABLE
F4STD-Q1,STD-Q,Queen,4,125,AVAILABLE
F4DLX-Q1,DLX-Q,Queen,4,159,OCCUPIED
F4DLX-T1,DLX-T,Twin,4,167,AVAILABLE
F4FAM1,FAM,Family,4,200,AVAILABLE

```